C++ Cheatsheet

Daniel Sinkin

March 8, 2026

Contents

1 Introduction **5**

2 Changes per version **5**
2.1 C++26 . . . 5
2.1.1 Contracts . . . 5
2.1.2 Anonymous values . . . 5
2.2 C++23 . . . 5
2.2.1 std::expected . . . 5
2.2.2 std::print, std::println . . . 6
2.2.3 std::generator . . . 6
2.3 C++20 . . . 6
2.3.1 std::format . . . 6
2.3.2 Concepts . . . 6
2.3.3 Ranges . . . 6
2.3.4 Coroutines . . . 6
2.3.5 Modules . . . 7
2.3.6 Calendar support for std::chrono . . . 7
2.4 C++17 . . . 7
2.4.1 CTAD (Class Template Argument Deduction) . . . 7
2.4.2 std::scoped_lock . . . 7
2.5 C++14 . . . 7
2.5.1 std::exchange . . . 7
2.5.2 constexpr loops and conditionals . . . 8
2.6 C++11 . . . 8
2.6.1 Value Semantics . . . 8
2.6.2 constexpr . . . 8
2.6.3 Lambdas . . . 8
2.6.4 std::unordered_map . . . 9
2.6.5 std::lock_guard . . . 9

3 C++ Concepts **10**
3.1 Value Semantics . . . 10
3.1.1 std::move . . . 10
3.1.2 Perfect Forwarding . . . 11
3.2 Access Control is Per-Class, Not Per-Instance . . . 11
3.3 Resource Ownership . . . 12
3.3.1 Rule of Five . . . 12
3.3.2 Rule of Zero . . . 12
3.3.3 RAII (Resource Acquisition is Initialisation) . . . 12
3.4 NRVO (Named Return Value Optimisation) . . . 13
3.5 EBO (Empty Base Optimisation) . . . 14
3.6 Idioms and Patterns . . . 14
3.6.1 Two-pointer merge . . . 14
3.6.2 Lazy Delete Idiom / Tombstoning . . . 15
3.6.3 Scope Guard / Defer Pattern . . . 15
3.6.4 Unsigned reverse iteration idiom (`i-- > 0`) . . . 15

3.6.5 Placement New . . . 16
3.6.6 std::construct_at, std::destroy_at (C++20) . . . 16
3.6.7 std::launder (C++17) . . . 16
3.7 ODR (One Definition Rule) . . . 17
3.8 SIOF (Static Initialisation Order Fiasco) . . . 17
3.9 Important STL algorithms . . . 17
3.10 Templates . . . 17
3.10.1 Variadic Templates . . . 17
3.10.2 Template Metaprogramming . . . 17
3.10.3 Concepts (C++20) . . . 17
3.10.4 Composing Concepts . . . 18
3.10.5 requires Expressions vs requires Clauses . . . 18
3.10.6 Partial Specialisation with Concepts . . . 19

4 STL (Standard Template Library) 20
4.1 Iterators . . . 20
4.1.1 Iterator Categories . . . 20
4.2 Ranges . . . 20
4.2.1 Core Concepts . . . 20
4.2.2 Views . . . 20
4.2.3 Range Algorithms . . . 20
4.2.4 Range Concepts . . . 20
4.3 Containers . . . 20
4.4 Algorithms . . . 20

5 Data Structures 21
5.1 std::vector (Trivially Copyable Types) . . . 21
5.1.1 Rule of 5 . . . 21
5.1.2 Public API . . . 22
5.1.3 Private API . . . 23
5.2 std::vector (General) . . . 23
5.3 std::unique_ptr . . . 23
5.3.1 Rule of 5 . . . 23
5.3.2 Public API . . . 24
5.3.3 Custom Deleter . . . 24
5.3.4 Make Unique . . . 25
5.4 std::optional . . . 25
5.4.1 Small Type Specialisation (inline storage) . . . 25
5.4.2 Large Type Specialisation (heap storage) . . . 26
5.5 Tree . . . 27
5.6 Binary Tree . . . 28
5.7 Complete Binary Tree . . . 28
5.8 std::priority_queue (Max Heap) . . . 28
5.9 std::unordered_map . . . 29
5.9.1 Open Addressing Hash Map Implementation . . . 29
5.10 Binary Search Tree (BST), Binary Search . . . 32
5.11 Red-Black Tree . . . 32
5.12 std::map . . . 32
5.13 std::array . . . 32
5.14 std::string . . . 32
5.15 std::deque . . . 32
5.16 std::pmr::memory_resource . . . 32
5.17 std::weak_ptr, std::shared_ptr, Control Block . . . 32
5.17.1 shared_ptr Implementation . . . 32
5.17.2 Constructors and Null-Safety . . . 33
5.17.3 Copy Operations (Must Lock the Source) . . . 33
5.17.4 Move Operations (Do NOT Lock) . . . 34
5.17.5 release() — Local Copy, Decrement Under Lock, Delete Outside . . . 34
5.17.6 reset() and Observers . . . 35

5.17.7 Implementation Pitfalls Summary . . . 35
5.18 union . . . 35
5.19 std::any (Type Erasure) . . . 36
5.19.1 Type Traits for SOO . . . 36
5.19.2 Class Layout . . . 36
5.19.3 Templated Constructor (Type Erasure Point) . . . 37
5.19.4 Destructor and any_cast . . . 37
5.20 std::variant . . . 37

6 Algorithms **38**
6.1 DFS (Depth First Search) . . . 38
6.2 BFS (Breadth First Search) . . . 38

7 Interview Specifics **39**
7.1 Tricks . . . 39
7.1.1 Custom Hasher . . . 39
7.1.2 2 Pointer Palindrome iteration . . . 39
7.1.3 Reverse Heap for bounded count . . . 39
7.1.4 Power-of-Two Division and Modulo via Bit Tricks . . . 41
7.1.5 Debug Macro . . . 41
7.1.6 Rolling Average . . . 42
7.1.7 Partition-based Binary Search (Two Sorted Arrays Median) . . . 43
7.1.8 Prefix Sums . . . 43
7.2 Problems . . . 44
7.2.1 Longest common subsequence . . . 44
7.2.2 Longest increasing subsequence . . . 44
7.2.3 Unique Paths . . . 45
7.2.4 Merge sorted linked lists . . . 46
7.2.5 Rotting Fruit . . . 47
7.3 Clone Graph . . . 48
7.3.1 Maximum Subarray . . . 49
7.3.2 Maximum Product Subarray . . . 49
7.3.3 Median of Two Circularly Sorted Logs . . . 49
7.3.4 Lazy Deletion Stack . . . 52
7.3.5 Subarrays with Given Sum and Bounded Maximum . . . 53
7.4 Maximize Profit with Task Deadlines and Multiple Servers . . . 54

8 Software Engineering **56**
8.1 Testing, Unit Tests, TDD (Test Driven Development) . . . 56
8.2 Design Patterns . . . 56
8.2.1 Visitor Pattern . . . 56
8.2.2 Strategy Pattern . . . 56
8.2.3 Factory Method . . . 56
8.2.4 Abstract Factory . . . 57
8.2.5 CRTP (Curiously Recurring Template Pattern) Design Pattern . . . 57
8.2.6 Type Erasure Design Pattern . . . 58

9 Databases **59**
9.1 MySQL . . . 59
9.2 MongoDB . . . 59

10 Computer Architecture, HPC (High Performance Computing) **60**
10.1 Memory Hierarchy and Cache . . . 60
10.1.1 Cache Lines and Cache Locality . . . 60
10.1.2 Spatial vs Temporal Locality . . . 60
10.1.3 False Sharing . . . 60
10.2 Memory Layout and Alignment . . . 61
10.2.1 Alignment vs Size . . . 61
10.2.2 Tight Packing vs Padding . . . 61

10.2.3 AoS vs SoA vs AoSoA . . . 61
10.2.4 SIMD Alignment Requirements . . . 62
10.2.5 Practical Trade-offs (Bandwidth vs Compute) . . . 62
10.3 CPU Microarchitecture . . . 62
10.3.1 Pipelining . . . 62
10.3.2 Branch Prediction and Speculative Execution . . . 62
10.3.3 Out-of-Order Execution . . . 62
10.4 GPU Architecture . . . 62
10.4.1 Streaming Multiprocessor (SM) and Warp Execution . . . 62
10.4.2 GPU Memory Hierarchy . . . 64
10.4.3 Memory Coalescing . . . 64
10.4.4 Shared Memory Bank Conflicts . . . 64
10.5 CUDA Programming Model . . . 64
10.5.1 Kernel Syntax and Launch Configuration . . . 65
10.5.2 2D Grid Addressing . . . 65
10.5.3 Block Size Selection . . . 65
10.5.4 Shared Memory and Synchronisation . . . 65
10.5.5 Tiled Matrix Multiplication . . . 66
10.6 CPU–GPU Interaction . . . 66
10.6.1 Kernel Launch Pipeline . . . 66
10.6.2 Asynchronous Execution . . . 66
10.7 Benchmarking and Measurement . . . 67
10.7.1 Sampling Benchmarks . . . 67
10.7.2 Cache and Memory Profiling (perf, Valgrind) . . . 67
10.7.3 Nsight Compute (GPU Profiling) . . . 67

11 Concurrency **68**
11.1 C++ Memory Model . . . 68
11.1.1 Atomics . . . 68
11.1.2 Memory Ordering (`std::memory_order`) . . . 68
11.2 Thread Utilities (`std::this_thread`) . . . 68
11.3 Mutexes and Locks . . . 68
11.3.1 Mutex Types . . . 68
11.3.2 Lock Types . . . 69
11.3.3 Condition Variables . . . 69
11.4 Multiple Threads . . . 70
11.5 Multiple Processes . . . 70
11.5.1 OpenMP . . . 70
11.6 Concurrent Data Structures . . . 70
11.6.1 Mutex (Spinlock) . . . 70
11.6.2 SPSC (Single Producer Single Consumer) . . . 71
11.6.3 Thread Pool . . . 72
11.6.4 SPMC (Single Producer Multiple Consumer) . . . 73
11.6.5 MPSC (Multiple Producer Single Consumer) . . . 73
11.6.6 MPMC (Multiple Producer Multiple Consumer) . . . 73

12 Operating Systems **74**
12.1 Process, Thread . . . 74
12.2 Scheduling . . . 74
12.3 CPU Virtualisation . . . 74
12.4 Memory Virtualisation . . . 74

13 Networking **75**
13.1 OSI Model . . . 75
13.2 UDP . . . 75
13.3 TCP/IP . . . 75

14 Trivia **76**
14.1 Error #323 on GCC . . . 76

1 Introduction

This document contains my notes on C++ specifics I'm reviewing for my job application.

2 Changes per version

2.1 C++26

2.1.1 Contracts

Will introduce Contracts which give an explicit syntax to implement post and pre conditions (formalising `gsl::Expects` and `gsl::Ensures`) as well as a new more stable assert syntax in form of `contract_assert`.

Listing 1: Pre-C++26: gsl::Expects/gsl::Ensures + assert

```
auto div_round_down_pos(std::int32_t a, std::int32_t b) -> std::int32_t
{
    gsl::Expects(a > 0);
    gsl::Expects(b > 0);

    const std::int32_t r = a / b;
    assert((r + 1) * b > a);

    Ensures(r * b <= a);
    return r;
}
```

Listing 2: C++26: Contract syntax

```
auto div_round_down_pos(std::int32_t a, std::int32_t b) -> std::int32_t
    pre  (a > 0)
    pre  (b > 0)
    post (r * b <= a)
{
    contract_assert((r + 1) * b > a);
    const std::int32_t r = a / b;
    return r;
}
```

2.1.2 Anonymous values

Sometimes we don't care about the name of a variable but still have to assign it, in the past you'd need to make up a (unique) name for the variable, now you can just use `_`.

Listing 3: C++26: Anonymous values

```
auto& [_, value] = f(); // Structured binding

Mutex m1{}, m2{};
{
    std::scoped_lock _{m1, m2}; // RAII utils
}
```

2.2 C++23

2.2.1 std::expected

Allows for functional / Rust style errors as values.

Listing 4: C++23: std::expected

```
enum class MyError {
    negative_number,
    zero_division,
```

```
};

std::expected<float, MyError> my_div(float a, float b) {
    if((a * b) <= 0.0f) {
        return std::unexpected{MyError::negative_number};
    }
    if(b == 0.0f) {
        return std::unexpected{MyError::zero_division};
    }
    return a / b;
}

int main() {
    auto res = my_div(5.0f, 3.0f);
    if(!res) {
        switch(res.error()) {
            case MyError::negative_number: {/*...*/}
            case MyError::zero_division: {/*...*/}
        }
    } else {
        float value{*res};
        /* ... */
    }
}
```

2.2.2 std::print, std::println

Introduced `print` which allows for structured printing by leveraging the C++20 feature `std::format`.

Listing 5: std::println

```
std::println("Hello, {}. The value of x is {}", "World", 5);
```

2.2.3 std::generator

`https://www.youtube.com/watch?v=7ZazVQB-RKc`

Can access like normal iterators

2.3 C++20

2.3.1 std::format

Introduced `std::format` which allows for formatting of variables into strings.

Listing 6: std::format

```
float x = 12.52343232f;
std::string s{std::format("x = {:.2f}", x)}; // s == "x = 12.52"
```

2.3.2 Concepts

Compile time constraints on templates, reducing the need for SFINAE boilerplate

Listing 7: Concepts

```
template <std::integral T>
T add(T a, T b) { return a + b; }
```

2.3.3 Ranges

2.3.4 Coroutines

```
co_await task;
co_yield value;
```

2.3.5 Modules

Adoption of those is horrible so far.

2.3.6 Calendar support for std::chrono

Listing 8: C++20: Calendar support for std::chrono

```
using namespace std::chrono;
auto zt = zoned_time{"Europe/Berlin", system_clock::now()};
```

2.4 C++17

2.4.1 CTAD (Class Template Argument Deduction)

Made template type deduction possible for class templates.

Listing 9: Pre-C++17: std::array without CTAD

```
#include <array>

std::array<int, 3> values{{1, 2, 3}};
```

Listing 10: C++17: std::array with CTAD

```
#include <array>

std::array values{1, 2, 3}; // std::array<int, 3>
```

2.4.2 std::scoped_lock

An improvement on `std::lock_guard` which allows to lock multiple mutexes in one call.

Listing 11: std::scoped_lock

```
std::mutex m1, m2;
{
    std::scoped_lock locks{m1, m2};
}
```

2.5 C++14

2.5.1 std::exchange

`std::exchange(obj, new_val)` assigns `new_val` to `obj` and returns the old value. It is a one-directional swap (a full swap is two exchanges). Not atomic. Particularly useful in move operations to transfer-and-null in a single expression, avoiding the window where both objects hold the same pointer.

Listing 12: std::exchange in move operations

```
// Without exchange: copy, then null (two steps)
ptr_ = other.ptr_;
other.ptr_ = nullptr;

// With exchange: one expression
ptr_ = std::exchange(other.ptr_, nullptr);

// Equivalent implementation:
template <class T, class U = T>
constexpr T exchange(T& obj, U&& new_val) {
    T old = std::move(obj);
    obj = std::forward<U>(new_val);
    return old;
}
```

2.5.2 constexpr loops and conditionals

Added support for loops and if/else branches.

Listing 13: C++14: constexpr with loops and conditionals

```
constexpr int sum_up_to(int n)
{
    int result = 0;

    for (int i = 1; i <= n; ++i)   // constexpr loop (C++14)
    {
        if (i % 2 == 0)            // constexpr conditional (C++14)
        {
            result += i;
        }
    }

    return result;
}
```

2.6 C++11

2.6.1 Value Semantics

Added move semantics and rvalue references.

Listing 14: Move constructors, Move assignment

```
std::vector<int> a = make_vector();
```

Listing 15: RAII

```
struct File {
    File(const char* path);
    ~File();
    File(File&&);
    File& operator=(File&&);
};
```

2.6.2 constexpr

Allows for compile time execution of code, for example offloading computations to compile time. Support was very sparse at this point and got continually extended.

Listing 16: constexpr (initial form)

```
constexpr float k_pi = 3.14f;

constexpr int square(int x) {
    return x * x;
}
```

2.6.3 Lambdas

Listing 17: C++11 Lambda

```
auto f = [](int x) {
    return x * x;
};
```

2.6.4 std::unordered_map

2.6.5 std::lock_guard

Listing 18: Lock Guard

```
Mutex x;
{
    std::lock_guard<Mutex> guard{x};
}
```

3 C++ Concepts

3.1 Value Semantics

Every Expression belongs to one of the following value categories:

- L value
- PR Value (Pure R value)
- X Value (eXpiring value)

When it is an L value or a X value we call it a GL (General L) value, if it is a PR value or a X value we call it a R value. In particular X values are both GL and R values. The naming is a bit unfortunate, it should PL value and L value; or GR value and R value.

Names are inspired by the fact that L values sit on the "left side of assignments" and R values sit on the "right side of assignments" and are temporaries in that sense. More concretely a L value is something that has a set memory location (`&x` or more accurately `std::address_of(x)`) which can be accessed. An R value is a temporary that does not have an addressable memory location (due to temporary materialisation this is no longer true, but an analogy to think about). X values are GL values which represent expiring objects, so they currently have a fixed memory handle but that one has a "soon" ending lifetime.

Listing 19: Rule of 5

```cpp
int x = 5;
x; // This id-expression is an L value
(x); // L value
&x; // prvalue of type int*

1 + 1; // prvalue (of type int)

struct Foo {/*...*/}
Foo foo{}; // declaration not an expression, so no value category

foo; // lvalue
f(foo); // passes an L value
f(std::move(foo)); // X value
f(Foo{}); // passes an PR value
```

3.1.1 std::move

`std::move` is an unconditional cast to rvalue reference. It does not move anything — it just enables moving by producing an xvalue. The standard implementation uses a single forwarding reference overload with `std::remove_reference_t` to strip any reference from `T` before forming `T&&` (otherwise reference collapsing would give `T&` when `T` deduces to an lvalue reference).

Listing 20: std::move — single overload (standard approach)

```cpp
template <typename T>
constexpr std::remove_reference_t<T>&& move(T&& t) noexcept {
    return static_cast<std::remove_reference_t<T>&&>(t);
}
```

Alternatively, explicit overloads avoid the need for `remove_reference_t` because `T` always deduces to the plain type when the parameter is `T&` (not a forwarding reference):

Listing 21: std::move — two explicit overloads

```cpp
template <typename T>
constexpr T&& move(T& t) noexcept {
    return static_cast<T&&>(t);
}

template <typename T>
constexpr T&& move(T&& t) noexcept {
    return static_cast<T&&>(t);
}
```

3.1.2 Perfect Forwarding

Under the hood `std::forward` is implemented as a cast:

```
template <class T>
constexpr T&& forward(std::remove_reference_t<T>& t) noexcept
{
    return static_cast<T&&>(t);
}

template <class T>
constexpr T&& forward(std::remove_reference_t<T>&& t) noexcept
{
    static_assert(!std::is_lvalue_reference_v<T>,
                  "bad forward: cannot forward an rvalue as an lvalue");
    return static_cast<T&&>(t);
}
```

```
class Foo{ /*...*/ };

void kind(T) { println("1"); }
void kind(const T&) { println("2"); }
void kind(T&&) { println("3"); }

template <typename T>
void bad_forward(T &&arg)
{
    kind(arg);
}

template <typename T>
void good_forward(T &&arg)
{
    kind(std::forward<T>(arg));
}

int main()
{
    Foo f{};
    bad_forward(f);        // prints: kind(Foo&)
    bad_forward(Foo{});    // prints: kind(Foo&)
    good_forward(f);       // prints: kind(Foo&)
    good_forward(Foo{});   // prints: kind(Foo&&)
}
```

3.2 Access Control is Per-Class, Not Per-Instance

In C++ `private` means "private to the class," not "private to the instance." Any member function of a class can access private members of *any* instance of that same class. This is why copy and move constructors work at all — without this rule you could not read `other`'s internal state.

Listing 22: Per-class access control

```
class Foo {
public:
    Foo(const Foo& other) {
        // other.x_ is private, but accessible here
        // because we are inside a Foo member function
        x_ = other.x_;
    }

    void steal_from(Foo& victim) {
        x_ = std::exchange(victim.x_, 0); // also fine
    }
```

```
private:
    int x_{};
};
```

3.3 Resource Ownership

3.3.1 Rule of Five

If you implement any of the following then you should implement all of them. The reasoning behind that is that you implementing a non-standard constructor or destructor implies that you have some non-trivial resource you don't trust the compiler to manage properly (e.g. heap allocations, database connection, file handle). This used to be the Rule of Three but now we have move and copy assignment constructors since C++11.

Listing 23: Rule of 5

```
class Foo {
    Foo() {}
    ~Foo() {...} // Destructor
    Foo(const Foo&) {...} // Copy Constructor
    Foo(Foo&&) {...} // Move Constructor
    Foo& operator=(const Foo&) {...} // Copy assignment constructor
    Foo& operator=(Foo&&) {...} // Move assignment constructor
}
```

3.3.2 Rule of Zero

Given that implementing the Rule of Five is annoying and creates a lot of boilerplate one should avoid that whenever possible, that is the Rule of Zero.

3.3.3 RAII (Resource Acquisition is Initialisation)

In my opinion this is a terrible name and should instead be CADR (Constructor Acquires Destructor Releases). It is a type of scope based automatic resource cleanup, the main idea is that when you invoke the constructor you obtain some resource which you hold for the entire lifetime of that class and once that lifetime has passed (on leaving scope) the resource gets automatically released. This is for example how one can use `std::vector` to avoid having to manually call `new` and `delete`.

To employ this we define a class with a constructor which allocates memory on the heap and a destructor which frees this memory.

```
class RAII {
    RAII(std::string_view name) : name_(name), data_(static_cast<int*>(std::malloc(8 *
    sizeof(int)))) {
        std::println("Allocated Memory '{}'", name_);
    }
    ~RAII() {
        std::free(data_);
        std::println("Deallocated Memory '{}'", name_);
    }
private:
    std::string name_{};
    int* data_{};
};
```

we then can't leak memory anymore, even if exceptions get thrown:

```
auto func() -> void {
    std::println("Starting Function");
    RAII func_scope("Function Scope");
    std::println("Starting inner scope");
    {
        RAII inner_scope("Inner Scope");
    }
    std::println("Finished inner scope");
    std::println("Throwing exception");
```

```
    throw std::runtime_error("");

    std::println("Finished function");
}

int main() {
    std::println("Starting Program");
    try {
        func();
    } catch (...) {
        std::println("Caught exception");
    }
    std::println("Finishing Program");
}
/*
Starting Function
Allocated Memory 'Function Scope'
Starting inner scope
Allocated Memory 'Inner Scope'
Deallocated Memory 'Inner Scope'
Finished inner scope
Throwing exception
Deallocated Memory 'Function Scope'
Caught exception
Finishing Program
*/
```

`scoped_lock` and (the now outdated `lock_guard`) are examples of RAII wrappers for mutexes, `std::vector` is a RAII wrapper around dynamic memory.

3.4 NRVO (Named Return Value Optimisation)

To show that NRVO happens we construct a simple tracker class which prints out whenever a copy / move / normal construction happens.

```
using Data = std::array<int, 32>;
class TrackMemory {
public:
    TrackMemory() {
        std::println("Empty Constructor");
    }
    TrackMemory(const TrackMemory &other) : data_(other.data_) {
        std::println("Copy constructor");
    }
    TrackMemory(TrackMemory &&other) noexcept : data_(std::move(other.data_)) {
        std::println("Move constructor");
    }
private:
    Data data_{};
};
```

Now we want to show three different cases, one where NRVO can't happen (different named variables of the same type get returned)

```
auto no_nrvo(bool return_a) -> TrackMemory {
    TrackMemory a{}, b{};
    if (return_a) { return a; }
    return b;
}
```

one where we, according to the standard, might have NRVO (returning a single named variable)

```
auto possible_nrvo() -> TrackMemory {
    TrackMemory a{};
    return a;
```

```
}
```

and one case where we are forced to have RVO (case where we return an unnamed temporary)

```
auto forced_nrvo() -> TrackMemory {
    return TrackMemory{};
}
```

We then run those

```
int main() {
    std::println("Impossible NRVO:");
    auto tm = no_nrvo(0);
    // Empty Constructor
    // Empty Constructor
    // Move constructor
    std::println("\nPossible NRVO:");
    auto tm2 = possible_nrvo();
    // Empty Constructor
    std::println("\nForced NRVO:");
    auto tm3 = forced_nrvo();
    // Empty Constructor
}
```

3.5 EBO (Empty Base Optimisation)

```
struct alignas(16) EmptyAligned {};

class NoEBO {
private:
    double data_{};
    EmptyAligned empty_internals_{};
};

class WithEBO {
private:
    double data_{};
    [[no_unique_address]] EmptyAligned empty_internals_{};
};

static_assert(std::is_empty_v<EmptyAligned>);
static_assert(sizeof(EmptyAligned) == 16);
static_assert(alignof(EmptyAligned) == 16);
static_assert(sizeof(NoEBO) == 32);
static_assert(sizeof(WithEBO) == 16);
```

3.6 Idioms and Patterns

3.6.1 Two-pointer merge

https://en.wikipedia.org/wiki/Merge_algorithm

If you want to merge two streams or lists together with some inequality relation on the values you do this:

```
std::vector<int> a{/*...*/};
std::vector<int> b{/*...*/};
std::vector<int> c{};
c.reserve(a.size() + b.size());

auto it_a = a.begin();
auto it_b = b.begin();
while(it_a != a.end() && it_b != b.end()) {
    if(*it_a <= *it_b) {
        c.push_back(*it_a);
```

```
        ++it_a;
    } else {
        c.push_back(*it_b);
        ++it_b;
    }
}
while(it_a != a.end()) {
    c.push_back(*it_a);
    ++it_a;
}
while(it_b != b.end()) {
    c.push_back(*it_b);
    ++it_b;
}
```

3.6.2 Lazy Delete Idiom / Tombstoning

When you want to delete an object in the middle (or beginning) of a vector you first have to push that element to the end and shift everything to the left by 1 and then `pop_back`, this can be very expensive, especially if many objects have to be deleted.

A common trick to avoid this is to keep a list of tombstones which are boolean values denoting if that value is deleted, and when you access or iterate you simply skip those elements.

If you occasionally pop elements off the back it can be advantageous to just keep popping as long as there are tombstone objects on the top, or if you have downtime in your program you could do cleanup (batching the removal is much faster) or just keep the tombstoned values if the memory cost is not too high.

```
struct DeletableInt{
    int value{};
    bool is_deleted{false};
};

std::vector<DeletableInt> vec{};
vec.resize(100);
for(auto i = 0; i < vec.size(); ++i) {
    if(i & 1 == 0) {
        vec[i].is_deleted = true;
    }
}
```

3.6.3 Scope Guard / Defer Pattern

RAII wrappers can also be useful when trying to do some post-scope cleanup or operation, as a type of soft `defer`. For example I use

```
using Clock = std::chrono::steady_clock;
using TimePoint = Clock::time_point;
using Duration = std::chrono::duration<f64>;

ScopeTimer::ScopeTimer(std::string_view label) noexcept : label_(label), start_(Clock::now()) {}
ScopeTimer::~ScopeTimer() noexcept
{
    const auto dt = Clock::now() - start_;
    const auto seconds = std::chrono::duration<f64>(dt).count();
    std::println("{}: {:.3f} ms", label_, seconds * 1000.0);
}
```

to quickly time scopes (for example actual scoped or function invocations).

3.6.4 Unsigned reverse iteration idiom (`i-- > 0`)

When iterating backwards with an unsigned index (e.g. `std::size_t`), `i >= 0` is always true, and `i--` can underflow. The idiom

```
for (auto i = n; i-- > 0;)
```

means: compare the current value to 0, then decrement, but the loop body sees the decremented value, running for indices `n-1, ..., 0`.

Listing 24: Reverse loop for unsigned indices (safe)

```
for (std::size_t i = v.size(); i-- > 0; )
{
    use(v[i]);
}
```

The range (C++20) based alternative to this is to use

```
for (auto& x : std::views::reverse(v)) {/*...*/}
```

3.6.5 Placement New

Regular `new` both allocates memory *and* constructs an object. Placement new skips the allocation and constructs an object at a specific memory address that you provide. This is essential when you manage your own storage (e.g. an aligned byte buffer in `std::optional`, pool allocators, or `std::vector` internals).

Listing 25: Placement new and manual destruction

```
// Allocate raw storage (no T constructor runs)
alignas(T) std::byte buffer[sizeof(T)];

// Construct T into the buffer (::new to bypass any class-local overloads)
T* ptr = ::new (&buffer) T{args...};

// Use the object normally
ptr->do_something();

// Must manually destroy (no delete! we don't own the memory)
ptr->~T();
```

Key rules: the target memory must be correctly aligned (`alignas(T)`), you must manually call the destructor when done (since there is no `delete` to pair with placement new), and use `::new` (globally scoped) to avoid accidentally calling a class-specific `operator new` overload. Calling `delete` on a placement-new'd object is undefined behaviour — it would try to free memory that was never heap-allocated.

3.6.6 std::construct_at, std::destroy_at (C++20)

`std::construct_at` and `std::destroy_at` are wrappers around placement new and explicit destructor calls. They are functionally equivalent at runtime but are `constexpr`-friendly (placement new is not allowed in `constexpr` contexts). For non-constexpr code, `::new` and manual `->~T()` are fine.

Listing 26: construct_at and destroy_at vs raw equivalents

```
alignas(T) std::byte buf[sizeof(T)];

// Equivalent pairs:
::new (&buf) T{args...};                // placement new
std::construct_at(reinterpret_cast<T*>(buf), args...);  // C++20

reinterpret_cast<T*>(buf)->~T();        // manual destructor
std::destroy_at(reinterpret_cast<T*>(buf));             // C++20
```

3.6.7 std::launder (C++17)

When an object is constructed via placement new into raw storage (e.g. a `std::byte[]` buffer), the compiler does not know a valid `T` lives at that address — it only sees the original `std::byte` array. Under aggressive optimisation, the compiler may assume no `T` exists there and optimise reads away. `std::launder` tells the compiler "a valid object of this type exists at this address":

Listing 27: std::launder usage

```
alignas(T) std::byte buf[sizeof(T)];
::new (&buf) T{42};

// Without launder: technically UB, compiler may optimise incorrectly
T& bad  = *reinterpret_cast<T*>(buf);

// With launder: formally correct
T& good = *std::launder(reinterpret_cast<T*>(buf));
```

In practice, every major compiler does the right thing without `std::launder` for simple cases. But it is formally required by the standard when accessing an object constructed in storage of a different type. The `std::any` and `std::optional` implementations both need it when reading from inline byte buffers.

3.7 ODR (One Definition Rule)

Use the `extern` keyword if you want to declare a variable but not define it to avoid ODR. Since C++17 you can (and should) use inline for variables instead.

A very subtle ODR bug can occur when you compile some files with debug symbols set and some without.

3.8 SIOF (Static Initialisation Order Fiasco)

When you have two non-local objects (e.g. globals or function-local statics) which have dynamic initialisation (run code in their constructors) which live in different translation units (.cpp files) then the initialisation order between them is not specified (standard only guarantees it within one translation unit (top-to-bottom)).

Listing 28: a.cpp

```
extern std::string g_name;

std::string make_greeting() { return "Hello " + g_name; }

std::string g_greeting = make_greeting();
```

Listing 29: b.cpp

```
std::string g_name = "Daniel";
```

Because `g_greeting` depends on `g_name` it might not have been initialised when you define it, so you'd run into UB. Note that if `a.cpp` would start with `std::string g_name = "Steve"` it would be an ODR (one definition rule) violation not SIOF.

3.9 Important STL algorithms

3.10 Templates

3.10.1 Variadic Templates

3.10.2 Template Metaprogramming

3.10.3 Concepts (C++20)

Concepts are named compile-time predicates that constrain template parameters. They replace SFINAE for most use cases and produce much clearer error messages. A concept is defined with `concept` and evaluates to `true` or `false` at compile time.

Listing 30: Defining and using concepts

```
template <typename T>
concept Hashable = requires(T a) {
    { std::hash<T>{}(a) } -> std::convertible_to<std::size_t>;
};

template <typename T>
concept SmallType = (sizeof(T) <= sizeof(char*));
```

Concepts can be used in four equivalent ways to constrain a template:

Listing 31: Four ways to apply a concept

```
// 1. requires-clause after template parameters
template <typename T> requires Hashable<T>
void foo(T x);

// 2. trailing requires-clause
template <typename T>
void foo(T x) requires Hashable<T>;

// 3. constrained template parameter
template <Hashable T>
void foo(T x);

// 4. abbreviated function template (terse syntax)
void foo(Hashable auto x);
```

3.10.4 Composing Concepts

Concepts can be combined with `&&` and `||` to build layered constraints. Define a minimal concept first, then compose it with additional requirements for more specific contexts. This avoids duplicating `requires` expressions.

Listing 32: Composing concepts: PointerDeleter and ControlBlockDeleter

```
// Minimal: callable with T* returning void
template <typename D, typename T>
concept PointerDeleter = requires(D d, T* p) {
    { d(p) } -> std::same_as<void>;
};

// Composed: also needs to be moveable into the control block
template <typename D, typename T>
concept ControlBlockDeleter =
    std::move_constructible<D> && PointerDeleter<D, T>;
```

When a concept is used in place of `typename`, the first template parameter is filled implicitly by the constrained argument. These two declarations are equivalent:

Listing 33: Shorthand vs longhand concept application

```
// Shorthand: Deleter fills D implicitly
template <typename T, ControlBlockDeleter<T> Deleter = std::default_delete<T>>
class shared_ptr { /* ... */ };

// Longhand: explicit requires clause
template <typename T, typename Deleter = std::default_delete<T>>
    requires ControlBlockDeleter<Deleter, T>
class shared_ptr { /* ... */ };
```

3.10.5 requires Expressions vs requires Clauses

The double `requires` syntax is confusing but the two keywords serve different roles. The first `requires` introduces a *clause* (a constraint on the template). The second `requires` starts an *expression* (a compile-time test of what operations a type supports).

Listing 34: requires clause vs requires expression

```
// Named concept: requires expression defines it
template <typename T>
concept Addable = requires(T a, T b) {
    { a + b } -> std::same_as<T>;
};

```

```
// Using the concept in a clause (single requires)
template <typename T> requires Addable<T>
T sum(T a, T b) { return a + b; }

// Inline without naming the concept (double requires)
template <typename T>
    requires requires(T a, T b) {
        { a + b } -> std::same_as<T>;
    }
T sum(T a, T b) { return a + b; }
```

The double `requires` form is rarely worth using inline — extracting a named concept is almost always more readable.

3.10.6 Partial Specialisation with Concepts

Partial specialisation allows a class template to have different implementations for different subsets of its type parameters. Since C++20, `requires`-clauses can be used to select between partial specialisations based on concept constraints. The compiler picks the most constrained matching specialisation.

The pattern is: declare a primary template (which can be left incomplete), then provide partial specialisations guarded by complementary `requires`-clauses. This is used in the `optional` implementation to select inline vs heap storage based on type size.

Listing 35: Constrained partial specialisation pattern

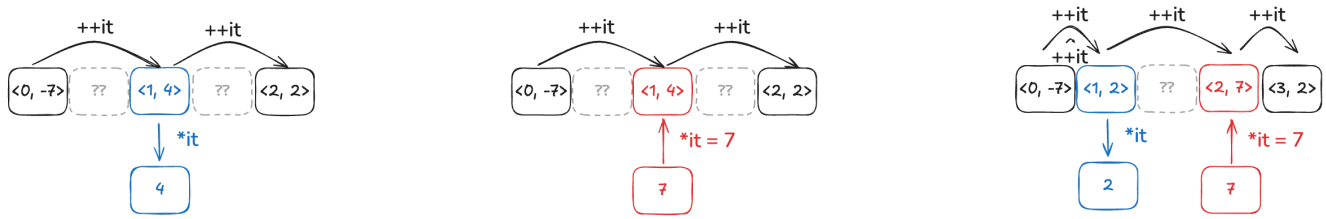
```
// Primary template: declared but not defined
template <typename T>
class Widget;

// Specialisation for small types: inline storage
template <typename T>
    requires(SmallType<T>)
class Widget<T> {
    alignas(T) std::byte data_[sizeof(T)];
};

// Specialisation for large types: heap storage
template <typename T>
    requires(!SmallType<T>)
class Widget<T> {
    T* data_{};
};

static_assert(sizeof(Widget<int>) <= 16);  // inline
// Widget<std::array<int, 1024>> uses heap
```

The key rules for partial specialisation: the specialisations must be strictly more constrained than the primary template, and for any given `T` exactly one specialisation must match (otherwise ambiguity is a compile error). Negating a concept with `!` ensures the two branches are mutually exclusive.

4 STL (Standard Template Library)

4.1 Iterators

4.1.1 Iterator Categories

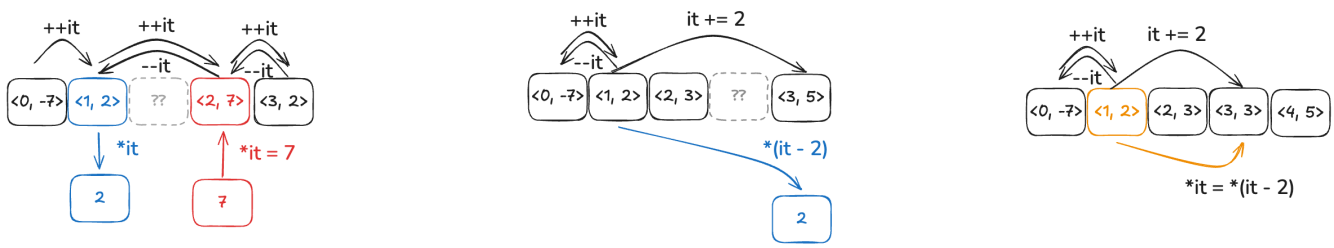


Input iterator

Output iterator

Forward iterator

Figure 1: Single-pass and forward iterators



Bidirectional iterator

Random-access iterator

Contiguous iterator

Figure 2: Bidirectional and stronger iterator categories

4.2 Ranges

First introduced in C++20, based on Eric Niebler's `Range-v3` library.

4.2.1 Core Concepts

4.2.2 Views

`std::ranges::filter`, `std::ranges::take`, `std::ranges::reverse`

For example the Unsigned reverse iteration idiom (`i-- > 0`) can be implemented using ranges instead:

```
for (auto& x : std::views::reverse(v)) {/*...*/}
```

4.2.3 Range Algorithms

`std::ranges::sort`, `std::ranges::find`, `std::ranges::copy`

4.2.4 Range Concepts

4.3 Containers

4.4 Algorithms

5 Data Structures

5.1 std::vector (Trivially Copyable Types)

A `vector<T>` is a heap allocated sequence container with three pointers `start_`, `end_`, `capacity_`. The first denotes the start of the vector, the second the location AFTER(!) the last allocated elements and the third is the total memory we have allocated. They take up 24 bytes (`3 * sizeof(T*)`) of stack memory and `(capacity_ - start_) * sizeof(T)` bytes of heap memory.

The core operation on a vector is `push_back` which takes an object and either inserts it in the back if there is space (`end_ != capacity_`) or first resizes (doubling capacity on GCC and Clang, increasing it by 1.5 on MSVC) and then inserting in the end.

If you want to delete an element you swap that position with `end_` and reduce `end_` by one.

This is a famously badly named datastructure as it squats on the name of mathematical elements of vector spaces and physics vectors. A better name would for example be DynamicArray.

Listing 36: Vector class

```cpp
template <typename T>
class VectorTrivial
{
    static_assert(std::is_trivially_copyable_v<T>);

public:
    using size_type = std::size_t;
    /*Rule of 5*/
    /*Public API*/
private:
    T* start_{};
    T* end_{};
    T* cap_{};
    /*Private API*/
};
```

5.1.1 Rule of 5

Listing 37: Rule of 5

```cpp
VectorTrivial() = default;
explicit VectorTrivial(size_type n)
{ // Allocate with fixed capacity
    start_ = static_cast<T*>(std::malloc(n * sizeof(T)));
    if(!start_) {/*...*/}
    end_ = start_;
    cap_ = start_ + n;
}

VectorTrivial(const Vector& other)
{ // Copy Constructor
    const auto n_elems = other.size();
    const auto n_capacity = other.capacity();

    start_ = static_cast<T*>(std::malloc(n_capacity * sizeof(T)));
    if(!start_) {/*...*/}
    std::memcpy(start_, other.start_, n_elems * sizeof(T));
    end_ = start_ + n_elems;
    cap_ = start_ + n_capacity;
}

VectorTrivial(Vector&& other) noexcept
    : start_(other.start_), end_(other.end_), cap_(other.cap_)
{ // Move Constructor
    other.start_ = nullptr;
    other.end_ = nullptr;
```

```
    other.cap_ = nullptr;
}

VectorTrivial& operator=(const Vector& other)
{ // Copy Assignment Constructor
    if(this == &other)
    {
        return *this;
    }

    const auto n_elems = other.size();
    const auto n_capacity = other.capacity();

    auto new_start = static_cast<T*>(std::malloc(n_capacity * sizeof(T)));
    if(!new_start) { /* ... */ }

    std::memcpy(new_start, other.start_, n_elems * sizeof(T));

    std::free(start_);
    start_ = new_start;
    end_ = start_ + n_elems;
    cap_ = start_ + n_capacity;
    return *this;
}

VectorTrivial& operator=(Vector&& other) noexcept
{ // Move Assignment Constructor
    if(this == &other)
    {
        return *this;
    }

    std::free(start_);
    start_ = other.start_;
    end_ = other.end_;
    cap_ = other.cap_;

    other.start_ = nullptr;
    other.end_ = nullptr;
    other.cap_ = nullptr;
    return *this;
}
~VectorTrivial()
{ // Destructor
    std::free(start_);
}
```

5.1.2 Public API

```
[[nodiscard]] auto size() const noexcept -> size_type
{
    return static_cast<size_type>(end_ - start_);
}
[[nodiscard]] auto capacity() const noexcept -> size_type
{
    return static_cast<size_type>(cap_ - start_);
}
[[nodiscard]] auto empty() const noexcept -> bool
{ // More efficient than checking size() == 0
    return end_ == start_;
}
void push_back(const T& v)
{ // push copy of element to the back, growing if necessary
```

```
    if (end_ == cap_) { grow_(); }
    *end_ = v;
    ++end_;
}
void push_back(T&& v)
{ // take ownership of element and push it to the back, growing if necessary
    if (end_ == cap_) { grow_(); }
    *end_ = std::move(v);
    ++end_;
}
```

5.1.3 Private API

```
// 1.5 on MSVC, 2.0 on GCC and Clang
constexpr double k_resize_factor = 2.0;
constexpr size_type k_initial_capacity = 8;
{
    const auto n_elems = size();
    const auto old_cap = capacity();
    const auto new_cap = (old_cap == 0)
        ? k_initial_capacity
        : static_cast<size_type>(old_cap * k_resize_factor);

    void* new_mem = std::realloc(start_, new_cap * sizeof(T));
    if (!new_mem) { /* handle allocation failure */ }

    start_ = static_cast<T*>(new_mem);
    end_ = start_ + n_elems;
    cap_ = start_ + new_cap;
}
```

5.2 std::vector (General)

5.3 std::unique_ptr

A unique pointer is a type of smart pointer which semantically has unique ownership over a pointer. It allows for arbitrary types and supports custom deleters (and therefore different types of resources, not only heap memory).

There is a free function which can create a unique pointer with its content inplace called `make_unique` which uses perfect forwarding to push arguments into the constructor of your underlying type using a variadic template.

Because it (by definition) managed non-trival resources it must implement the Rule of 5. The destructor should automatically invoke the deleter in its destructor.

It must expose a way to access the underlying data, and it must be able to release / relinquish its control over the data it holds.

```
template <class T, class Deleter = DefaultDeleter<T>>
class UniquePtr {
public:
    using pointer_type = T*;
    /* Rule of 5*/
private:
    // Trick to make it zero cost abstraction
    [[no_unique_address]] Deleter deleter_{};
    pointer_type ptr_{};
};
```

5.3.1 Rule of 5

A unique pointer is responsible for cleaning up the underlying resource (as it is a RAII wrapper), we should be able to move one unique pointer into another to move ownership, but copying a unique pointer is meaningless so copy constructor and copy assignment constructor both get deleted.

```cpp
// Take ownership over a pointer
UniquePtr(pointer_type ptr) { ptr_ = ptr; }

// Copying is disallowed
UniquePtr(const UniquePtr&) = delete;
UniquePtr& operator=(const UniquePtr&) = delete;

// Moving is allowed
UniquePtr(UniquePtr&& other) noexcept(/*Deleter must be noexcept moveable and move constructible*/)
    : ptr_(other.release()), deleter_(std::move(other.deleter_)) {}

UniquePtr& operator=(UniquePtr&& other) {
    if(this == &other) {
        return *this;
    }
    if(ptr_) {
        deleter_(ptr_);
    }

    ptr_ = other.release();
    deleter_ = std::move(other.deleter_);
    return *this;
}

~UniquePtr() {
    if(ptr_) {
        deleter_(ptr_);
    }
}
```

5.3.2 Public API

It should be possible to `get` the underlying data of the pointer and for the pointer to `release` its ownership to the memory.

```cpp
auto release() -> pointer_type {
    auto ptr = ptr_;
    ptr_ = nullptr;
    return ptr;
}

auto get() const -> pointer_type {
    return ptr_;
}
```

5.3.3 Custom Deleter

For the unique pointer to be able to manage different types of resources it is important to offer an (optional) custom deleter, but also provide a default deleter (equivalent to `std::default_delete`). Of course that has to be templated over the underlying type. There must be two overloads, one for arrays and one for non-arrays.

```cpp
template <class T>
struct DefaultDelete {
    auto operator()(T* p) -> void {
        delete p;
    }
}

template <class T>
struct DefaultDelete<T[]> {
    auto operator()(T* p) -> void {
        delete[] p;
    }
```

```
}
```

5.3.4 Make Unique

A free function which constructs the object and unique pointer together to avoid having to directly construct the object and then move it into a pointer. It is a variadic template that forwards arguments to the constructor of the underlying type. There are three different cases to consider, non-array pointers, array pointers with unknown bounds and array pointers with known bounds. We only want to support the first two.

```
template <class T, class...Args>
    requires (!std::is_array_v<T>)
auto make_unique(Args&&...args) -> UniquePtr<T> {
    return UniquePtr<T>(new T(std::forward<Args>(args)...));
}

template <class T, class...Args>
    requires (std::is_array_v<T> && std::extent_v<T> == 0)
auto make_unique(Args&&...args) -> UniquePtr<T> {
    using  U = std::remove_extent_t<T>;
    return UniquePtr<U>(new U[n]());
}

template <class T, class...Args>
    requires (std::is_array_v<T> /*&& bounds side > 0*/)
auto make_unique(Args&&...) -> UniquePtr<T> = delete;
```

5.4 std::optional

This is a container that stores a value or denotes its absence. The standard implementation stores the object inline via placement new into an aligned byte buffer, avoiding heap allocation.

This implementation uses constrained partial specialisation (see Templates section) to select between two strategies at compile time: small types ($\leq$ pointer size) are stored inline in an aligned byte buffer, while large types fall back to a heap pointer. Both specialisations share the same public interface but differ in storage layout. Copy and move are deleted for simplicity.

Listing 38: SmallType concept and primary template

```
template <typename T>
concept SmallType = (sizeof(T) <= sizeof(char*));

template <typename T>
class optional; // declared but not defined; only specialisations are valid
```

5.4.1 Small Type Specialisation (inline storage)

For types that fit in a pointer, we store the value directly in an aligned byte buffer. The skeleton shows the storage layout: an aligned byte array sized to `T` and a boolean flag.

Listing 39: optional<T> skeleton for SmallType

```
template <typename T>
    requires(SmallType<T>)
class optional<T> {
public:
    optional(const optional&) = delete;
    optional& operator=(const optional&) = delete;
    optional(optional&&) = delete;
    optional& operator=(optional&&) = delete;

    optional() = default;
    optional(T a);
    ~optional();

```

```
    [[nodiscard]] auto has_value() const noexcept -> bool;
    [[nodiscard]] operator bool() const noexcept;
    [[nodiscard]] auto value() const -> const T&;
    [[nodiscard]] auto operator*() const -> const T&;
private:
    alignas(T) std::byte data_[sizeof(T)];
    bool present_{false};
};
```

Constructor: Uses placement new to construct `T` directly into the byte buffer. No heap allocation occurs.

Listing 40: SmallType optional: constructor

```
optional(T a) {
    ::new (data_) T{a};
    present_ = true;
}
```

Destructor: Must manually invoke the destructor through a `reinterpret_cast` since the object lives in raw storage, not a typed variable.

Listing 41: SmallType optional: destructor

```
~optional() {
    if(present_) {
        reinterpret_cast<T*>(data_)->~T();
    }
}
```

Accessors: `has_value` and `operator bool` check the flag. `value` reinterprets the byte buffer back to `const T&`, throwing if empty. `operator*` delegates to `value`.

Listing 42: SmallType optional: accessors

```
[[nodiscard]] auto has_value() const noexcept -> bool {
    return present_;
}
[[nodiscard]] operator bool() const noexcept {
    return has_value();
}
[[nodiscard]] auto value() const -> const T& {
    if(!present_) {
        throw std::runtime_error("no value");
    }
    return *reinterpret_cast<const T*>(data_);
}
[[nodiscard]] auto operator*() const -> const T& {
    return value();
}
```

5.4.2 Large Type Specialisation (heap storage)

For types larger than a pointer, we heap-allocate. Presence is indicated by a non-null pointer, so no separate boolean is needed.

Listing 43: optional<T> skeleton for !SmallType

```
template <typename T>
    requires(!SmallType<T>)
class optional<T> {
public:
    optional(const optional&) = delete;
    optional& operator=(const optional&) = delete;
    optional(optional&&) = delete;
    optional& operator=(optional&&) = delete;

    optional() = default;
```

```
    optional(T a);
    ~optional();

    [[nodiscard]] auto has_value() const noexcept -> bool;
    [[nodiscard]] operator bool() const noexcept;
    [[nodiscard]] auto value() const -> const T&;
    [[nodiscard]] auto operator*() const -> const T&;
private:
    T* data_{};
};
```

Constructor and destructor: The constructor heap-allocates a copy. The destructor simply calls `delete`, which is a no-op on `nullptr` so no guard is needed.

Listing 44: Large optional: constructor and destructor

```
optional(T a) {
    data_ = new T{a};
}
~optional() {
    delete data_;
}
```

Accessors: Same interface as the small specialisation, but presence is determined by a null check instead of a boolean flag.

Listing 45: Large optional: accessors

```
[[nodiscard]] auto has_value() const noexcept -> bool {
    return data_ != nullptr;
}
[[nodiscard]] operator bool() const noexcept {
    return has_value();
}
[[nodiscard]] auto value() const -> const T& {
    if(!data_) {
        throw std::runtime_error("no value");
    }
    return *data_;
}
[[nodiscard]] auto operator*() const -> const T& {
    return value();
}
```

5.5 Tree

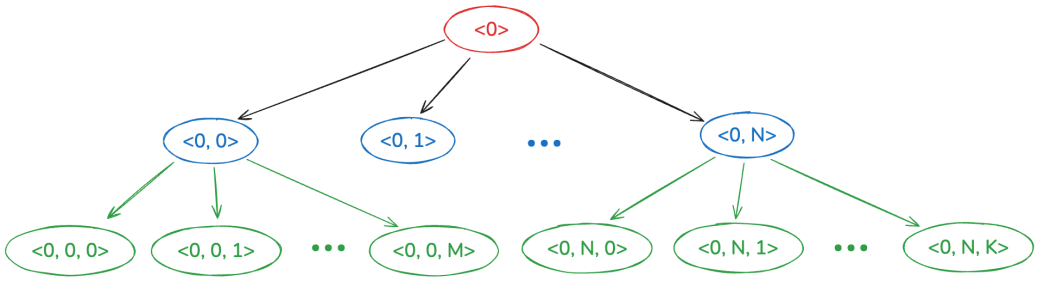


Figure 3: General tree (arbitrary number of children per node)

```
struct TreeNode {
    int value{};
    std::vector<std::unique_ptr<TreeNode>> children{};
};
```

5.6 Binary Tree

A special type of Tree where every node has at most 2 children.

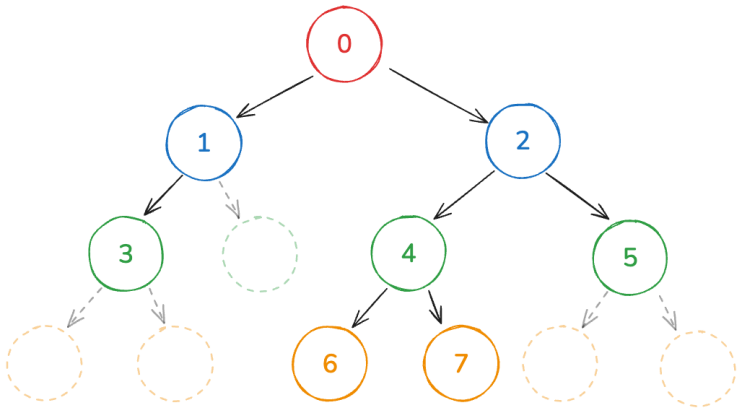


Figure 4: Binary tree (each node has at most two children)

```
struct BinaryTreeNode {
    int value{};
    std::unique_ptr<BinaryTreeNode> left{};
    std::unique_ptr<BinaryTreeNode> right{};
};
```

5.7 Complete Binary Tree

A complete binary tree is one that is "filled up" from left to right, meaning that each level gets filled in first if possible.

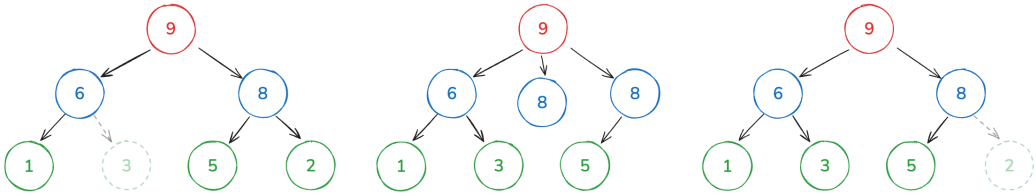


Figure 5: Left is a binary tree that is not complete, middle is not a binary tree, right is a complete binary tree

Complete binary trees can be addressed very efficiently by the rule that the element with offset i in level L is indexed by

$$I(L, i) = 2^L - 1 + i.$$

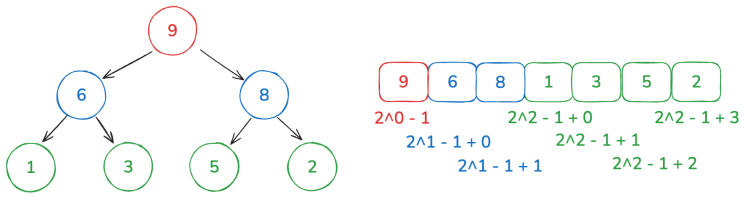


Figure 6: Complete binary tree (filled left-to-right)

5.8 std::priority_queue (Max Heap)

Suppose you want to efficiently pop the largest / smallest entry of a list, this is what max / min heaps are great for. Idea is that you have a linear array representing a complete binary tree in the usual way (root is idx 0, its children are 1, 2, their children are 3, 4 and 5, 6 respectively and so on).

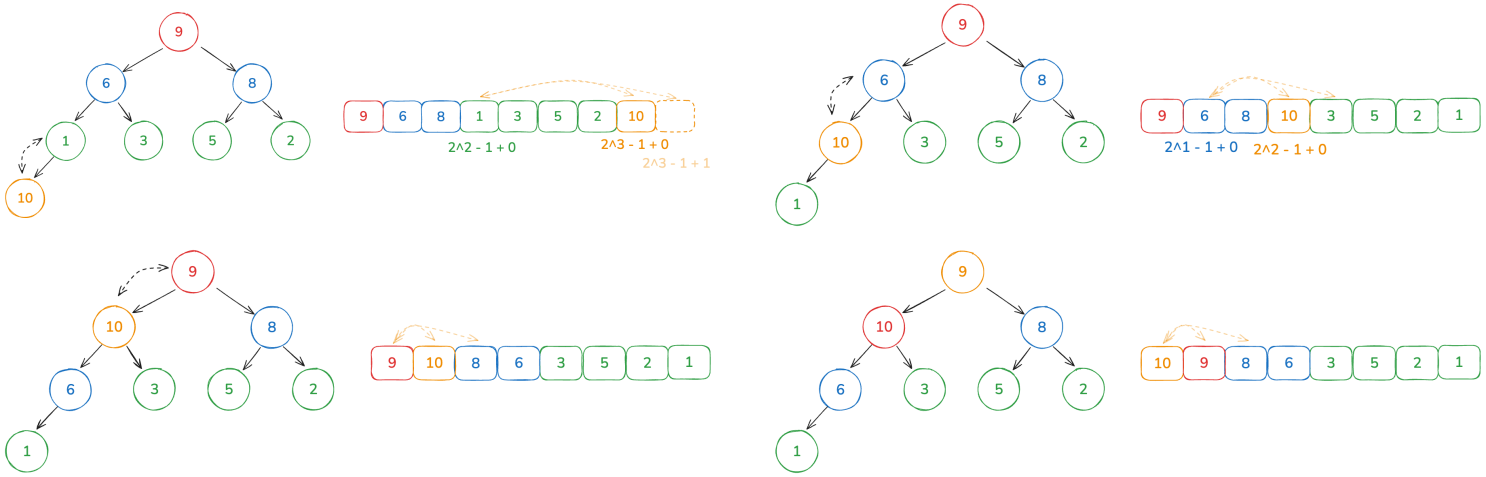


Figure 7: Max-heap insertion (heapify-up steps)

5.9 std::unordered_map

Hashmap, you can use your own hashing function.

5.9.1 Open Addressing Hash Map Implementation

An open addressing hash map stores all entries directly in the backing array (no linked lists). On collision, we probe forward (linear probing) until we find an empty slot. The capacity is always a power of two so we can replace modulo with a bitmask.

Each slot has metadata tracking its state: **empty** (never used), **filled** (holds a live key-value pair), or **tombstone** (was filled, then erased). Tombstones are necessary because linear probing relies on the invariant that a probe chain is contiguous; if we cleared a slot to empty on erase, a later lookup could stop too early and miss entries that were inserted after the erased one. A tombstone says "keep probing, something was here." Tombstones are skipped during lookup but can be overwritten during insert. They are cleaned up on rehash (only filled entries are re-inserted).

The map rehashes when the load factor exceeds 50% to keep probe chains short.

Listing 46: OpenAddressingHashMap skeleton

```cpp
enum class ESlotMetadata : u8 {
    empty = 0,
    filled = 1,
    tombstone = 2
};

template <Hashable Key, typename Value>
class OpenAddressingHashMap
{
public:
    using SizeT = std::uint32_t;

    OpenAddressingHashMap();

    auto insert(const Key& key, const Value& value) -> void;
    auto erase(const Key& key) -> bool;
    auto get(const Key& key) -> Value&;
    auto get(const Key& key) const -> const Value&;
    auto contains(const Key& key) const -> bool;
    auto get_size() const -> SizeT;
    auto get_capacity() const -> usize;
private:
    static constexpr usize k_initial_capacity = 16zu;

    std::vector<Value> data_;
    std::vector<ESlotMetadata> metadata_;
    std::vector<Key> keys_;
    SizeT num_elements_{0};
```

```

    static constexpr SizeT k_max_size =
        std::numeric_limits<SizeT>::max() / 2;

    auto mask() const noexcept -> SizeT;
    auto filled_with_key(const Key& key, SizeT idx) const noexcept -> bool;
    auto insert_no_resize(const Key& key, const Value& value) -> void;
    auto resize_and_rehash() -> void;
};
```

The constructor allocates the initial capacity and marks all slots as empty. Three parallel vectors hold values, keys, and metadata separately (structure-of-arrays layout).

Listing 47: Constructor

```
OpenAddressingHashMap()
{
    data_.resize(k_initial_capacity);
    keys_.resize(k_initial_capacity);
    metadata_.assign(k_initial_capacity, ESlotMetadata::empty);
}
```

`mask()`: Since capacity is always a power of two (2^k), the bit pattern is `100...00`. Subtracting 1 gives `011...11`; bitwise AND with this is equivalent to modulo but avoids the expensive division.

Listing 48: Bitmask index and key check

```
auto mask() const noexcept -> SizeT
{
    // (a % 2^k) == a & (2^k - 1)
    return static_cast<SizeT>(get_capacity() - 1);
}

auto filled_with_key(const Key& key, SizeT idx) const noexcept -> bool
{
    return metadata_[idx & mask()] == ESlotMetadata::filled
        && keys_[idx & mask()] == key;
}
```

`insert`: Checks load factor first and rehashes if above 50%. Then delegates to `insert_no_resize` which probes forward, skipping filled slots with different keys (but stopping on a match to overwrite). Tombstones are implicitly overwritten since the loop only continues on `filled` slots with non-matching keys.

Listing 49: insert and insert_no_resize

```
auto insert(const Key& key, const Value& value) -> void
{
    if (num_elements_ + 1 > data_.size() / 2) {
        resize_and_rehash();
    }
    insert_no_resize(key, value);
}

auto insert_no_resize(const Key& key, const Value& value) -> void
{
    auto idx = static_cast<SizeT>(std::hash<Key>{}(key));
    while (metadata_[idx & mask()] == ESlotMetadata::filled
           && keys_[idx & mask()] != key) {
        ++idx;
    }
    const auto was_already_filled = filled_with_key(key, idx);
    metadata_[idx & mask()] = ESlotMetadata::filled;
    data_[idx & mask()] = value;
    keys_[idx & mask()] = key;
    if (!was_already_filled) {
        ++num_elements_;
```

```
    }
}
```

erase: Probes forward from the hash index. If we find the key, we mark the slot as a tombstone (not empty) and decrement the count. Returning `false` if we hit an empty slot means the key was never inserted.

Listing 50: erase

```
auto erase(const Key& key) -> bool
{
    auto idx = static_cast<SizeT>(std::hash<Key>{}(key));
    while (metadata_[idx & mask()] != ESlotMetadata::empty) {
        if (filled_with_key(key, idx)) {
            metadata_[idx & mask()] = ESlotMetadata::tombstone;
            --num_elements_;
            return true;
        }
        ++idx;
    }
    return false;
}
```

get and contains: Same probe loop. `get` throws on miss; `contains` returns a bool.

Listing 51: get and contains

```
auto get(const Key& key) -> Value&
{
    auto idx = static_cast<SizeT>(std::hash<Key>{}(key));
    while (metadata_[idx & mask()] != ESlotMetadata::empty) {
        if (filled_with_key(key, idx)) {
            return data_[idx & mask()];
        }
        ++idx;
    }
    throw std::out_of_range("Key does not exist");
}

auto contains(const Key& key) const -> bool
{
    auto idx = static_cast<SizeT>(std::hash<Key>{}(key));
    while (metadata_[idx & mask()] != ESlotMetadata::empty) {
        if (filled_with_key(key, idx)) {
            return true;
        }
        ++idx;
    }
    return false;
}
```

resize_and_rehash: Doubles the capacity. Copies the old data, resets all metadata to empty, then re-inserts only `filled` entries (tombstones are discarded, cleaning up the table).

Listing 52: resize_and_rehash

```
auto resize_and_rehash() -> void
{
    if (data_.size() >= k_max_size) {
        throw std::runtime_error("Max Size overflow");
    }
    const auto old_size = data_.size();
    const auto new_size = 2 * old_size;

    const auto data_old = data_;
    const auto keys_old = keys_;
    const auto metadata_old = metadata_;
```

```

    data_.resize(new_size);
    keys_.resize(new_size);
    metadata_.assign(new_size, ESlotMetadata::empty);
    num_elements_ = SizeT{0};

    for (usize i{0zu}; i < old_size; ++i) {
        if (metadata_old[i] == ESlotMetadata::filled) {
            insert_no_resize(keys_old[i], data_old[i]);
        }
    }
}
```

5.10 Binary Search Tree (BST), Binary Search

A BST is a binary tree which satisfies the property that each node is greater or equal to all nodes in its left subtree and less than all nodes in its right subtree. It allows efficient searching for elements.

5.11 Red-Black Tree

A self-balancing BST.

5.12 std::map

This stores the keys as a (balanced) binary search tree (more specifically as a red-black tree).

5.13 std::array

5.14 std::string

5.15 std::deque

5.16 std::pmr::memory_resource

5.17 std::weak_ptr, std::shared_ptr, Control Block

Both `std::weak_ptr` and `std::shared_ptr` internally track objects via the control block which holds strong counter, weak counter, deleter and (optionally) an allocator.

```
struct ControlBlock {
    std::atomic<usize> strong{};
    std::atomic<usize> weak{};
    void (*deleter)(void*);
    void* ptr;
}
```

and `std::weak_ptr` holds a pointer to a ControlBlock, `std::shared_ptr` holds a pointer to the object and a ControlBlock.

5.17.1 shared_ptr Implementation

A thread-safe `shared_ptr` with a custom deleter (constrained by concepts), `std::exchange`-based moves, and the local-copy release pattern. The control block stores the reference count, a mutex, and the deleter (with `[[no_unique_address]]` to avoid overhead for stateless deleters). The `shared_ptr` itself holds a raw pointer to the managed object and a pointer to the control block.

Listing 53: Concepts and Class Layout

```
template <typename D, typename T>
concept PointerDeleter = requires(D d, T* p) {
    { d(p) } -> std::same_as<void>;
};

template <typename D, typename T>
concept ControlBlockDeleter =
```

```
    PointerDeleter<D, T> && std::move_constructible<D>;

template <typename T, ControlBlockDeleter<T> Deleter = std::default_delete<T>>
class shared_ptr {
private:
    struct ControlBlock {
        size_t count_{};
        mutable std::mutex mutex_;
        [[no_unique_address]] Deleter deleter_;
    };
public:
    // ...
private:
    T* ptr_{};
    ControlBlock* block_{};
};
```

5.17.2 Constructors and Null-Safety

The raw-pointer constructor should avoid allocating a control block when `p` is `nullptr`. This keeps the invariant that `block_ == nullptr` iff the pointer is empty. A `nullptr_t` constructor delegates to the default constructor. The deleter overload also null-checks and `std::move`s the deleter into the control block. Note: constructing with `nullptr` and a custom deleter silently loses the deleter (no control block is allocated), which matches STL behaviour.

Listing 54: Constructors

```
shared_ptr() = default;
explicit shared_ptr(std::nullptr_t) : shared_ptr() {}

explicit shared_ptr(T* pointer)
    : ptr_{pointer}
    , block_{pointer ? new ControlBlock{1, {}, {}} : nullptr}
{}

explicit shared_ptr(T* pointer, Deleter deleter)
    : ptr_{pointer}
    , block_{pointer ? new ControlBlock{1, {}, std::move(deleter)}
                     : nullptr}
{}
```

5.17.3 Copy Operations (Must Lock the Source)

Pitfall: In the copy constructor, `block_` on `this` is *uninitialised*. You must lock `other`'s control block, not your own — dereferencing an uninitialised control block is UB. Copy the pointers first, then lock and increment. Copy assignment must call `release()` first to drop the current resource.

Listing 55: Copy Constructor and Copy Assignment

```
shared_ptr(const shared_ptr& other) noexcept {
    if (!other) {
        ptr_   = nullptr;
        block_ = nullptr;
        return;
    }
    ptr_   = other.ptr_;
    block_ = other.block_;
    {
        std::scoped_lock sl{block_->mutex_};
        ++block_->count_;
    }
}

shared_ptr& operator=(const shared_ptr& other) noexcept {
    if (this == &other) return *this;
```

```
    release();
    if (!other) {
        ptr_   = nullptr;
        block_ = nullptr;
        return *this;
    }
    ptr_   = other.ptr_;
    block_ = other.block_;
    {
        std::scoped_lock sl{block_->mutex_};
        ++block_->count_;
    }
    return *this;
}
```

5.17.4 Move Operations (Do NOT Lock)

Moves assume sole ownership of the source. No other thread should be accessing the moved-from object, so locking is unnecessary (and dangerous if the control block has already been released). `std::exchange` transfers and nulls in a single expression, avoiding any window where both objects hold the same pointer.

Listing 56: Move Constructor, Move Assignment, and steal_from helper

```
shared_ptr(shared_ptr&& other) noexcept {
    steal_from(std::move(other));
}

shared_ptr& operator=(shared_ptr&& other) noexcept {
    if (this == &other) return *this;
    release();
    steal_from(std::move(other));
    return *this;
}

// private:
auto steal_from(shared_ptr&& other) -> void {
    ptr_   = std::exchange(other.ptr_, nullptr);
    block_ = std::exchange(other.block_, nullptr);
}
```

5.17.5 release() — Local Copy, Decrement Under Lock, Delete Outside

The correct pattern: decrement the count under the lock via an IIFE, then copy pointers to locals with `std::exchange` (nulling the members *before* deletion), and finally delete through the locals. This avoids both (a) destroying the mutex while held and (b) leaving dangling pointers on `this` during deletion (in case the deleter observes this shared_ptr).

Listing 57: release() and Destructor

```
auto release() -> void {
    if (!block_) return;
    const auto should_free = [&]{
        std::scoped_lock sl{block_->mutex_};
        return --block_->count_ == 0zu;
    }();
    ControlBlock* local_block = std::exchange(block_, nullptr);
    T* local_ptr = std::exchange(ptr_, nullptr);
    if (should_free) {
        local_block->deleter_(local_ptr);
        delete local_block;
    }
}

~shared_ptr() { release(); }
```

5.17.6 reset() and Observers

`reset()` with no arguments simply releases. `reset(T*)` null-checks the new pointer: if it is `nullptr`, do not allocate a control block. A deleter overload is also provided. The ref-count accessor must lock before reading. Observers use `[[nodiscard]]` and `operator bool` is `explicit` to prevent accidental implicit conversions in arithmetic contexts.

Listing 58: reset, get_count, get, operators

```cpp
auto reset() -> void { release(); }

auto reset(T* pointer) -> void {
    release();
    if (pointer) {
        ptr_   = pointer;
        block_ = new ControlBlock{1, {}, {}};
    }
}

auto reset(T* pointer, Deleter deleter) -> void {
    release();
    if (pointer) {
        ptr_   = pointer;
        block_ = new ControlBlock{1, {}, std::move(deleter)};
    }
}

[[nodiscard]] auto get_count() const -> std::size_t {
    if (!block_) return 0zu;
    std::scoped_lock sl{block_->mutex_};
    return block_->count_;
}

[[nodiscard]] auto get() const -> T* { return ptr_; }
[[nodiscard]] auto operator->() const -> T* { return ptr_; }
[[nodiscard]] auto operator*() const -> T& { return *ptr_; }
[[nodiscard]] explicit operator bool() const noexcept {
    return ptr_ != nullptr;
}
```

5.17.7 Implementation Pitfalls Summary

1. **Copy ctor locks source, not `this`** — `block_` on `this` is uninitialised; dereferencing it is UB.
2. **All non-void methods must return on every path** — `return;` in a method returning `shared_ptr&` won't compile. Always `return *this;` (watch for the `this*` typo).
3. **`operator*` bugs** — (a) inverted null check (`if(ptr_) throw` vs `if(!ptr_) throw`), (b) returning `&ptr_` (`T**`) instead of `*ptr_`.
4. **Delete outside the lock** — Destroying a mutex while it is held is UB. Use the IIFE-capture pattern shown in `release()`.
5. **Null members before deleting** — Copy pointers to locals with `std::exchange`, null the members, *then* delete through locals. Prevents dangling pointers if the deleter observes `this`.
6. **Null pointer constructor** — `shared_ptr(T* p)` should skip control-block allocation when `p == nullptr`.
7. **Enforce the `ptr_`/`block_` invariant** — `ptr_` and `block_` must always be both null or both non-null. At every early-return null check, `assert` that the other member is also null.
8. **`explicit` on raw-pointer constructor and `operator bool`** — prevents accidental implicit conversions from `T*` to `shared_ptr` and from `shared_ptr` to `int`.

5.18 union

A `union` is a type where all members share the same memory. The size of a union equals the size of its largest member. Only one member is "active" at a time — writing to one member makes the others' values undefined. This is the foundation of small object optimisation (SOO): you can overlay a small inline buffer with a heap pointer in the same storage, avoiding wasted space.

Listing 59: Basic union and anonymous union

```cpp
// Named union type:
union Storage {
    alignas(std::max_align_t) std::byte buffer[32];
    void* ptr;
};
Storage s{};
s.ptr = new int{42};  // 'buffer' is now inactive

// Anonymous union (members accessible directly):
class any {
    union {
        alignas(std::max_align_t) std::byte buffer_[32];
        void* heap_;
    };
    // buffer_ and heap_ are direct members of 'any'
};
```

Key rules: `alignas(std::max_align_t)` on the buffer ensures any standard type can be placement-new'd into it. When using a union with non-trivial types, you must manually manage construction and destruction (the compiler cannot generate constructors/destructors for unions with non-trivial members). A default member initialiser (e.g. `void* ptr{}`) determines which member is initially active.

5.19 std::any (Type Erasure)

`std::any` (C++17) is a type-safe container that holds a value of any type — like `void*` but with type information preserved. The type is erased at the class level (`any` is not templated) but remembered at runtime via `std::type_info`. Casting to the wrong type throws `std::bad_cast`.

The core technique: the templated *constructor* captures type-specific operations (destruction, copying) as function pointers or `std::function` objects. Once stored, the class invokes these without knowing `T`. This is called type erasure.

5.19.1 Type Traits for SOO

A custom trait selects between small (inline) and large (heap) storage. Unlike concepts, traits work with `if constexpr` inside a non-templated class.

Listing 60: Small object trait

```cpp
template <typename T>
struct is_small_trait {
    constexpr static inline bool value = false;
};

template <>
struct is_small_trait<int> {
    constexpr static inline bool value = true;
};

template <typename T>
constexpr bool is_small_v = is_small_trait<T>::value;
```

5.19.2 Class Layout

The union overlays inline storage with a heap pointer. A `std::function` stores the type-erased deleter, and a `std::type_info` pointer enables runtime type checking.

Listing 61: any class layout

```cpp
class any {
private:
    const std::type_info* type_{};

    union Storage {
        alignas(std::max_align_t)
            std::byte small_[4 * sizeof(void*)];
```

```
        void* large_{};
    };

    Storage storage_{};
    std::function<void(Storage&)> deleter_{};
};
```

5.19.3 Templated Constructor (Type Erasure Point)

The constructor is the only place where `T` is known. It stores the `type_info`, constructs the object (inline or heap), and captures the appropriate deleter as a lambda. The lambda captures `T` at compile time but is stored as a type-erased `std::function`.

Listing 62: any constructor

```
template <typename T>
any(T t) {
    type_ = &typeid(T);
    if constexpr (is_small_v<T>) {
        std::construct_at(
            reinterpret_cast<T*>(storage_.small_), t);
        deleter_ = [](Storage& s) {
            std::destroy_at(std::launder(
                reinterpret_cast<T*>(s.small_)));
        };
    } else {
        storage_.large_ = new T{t};
        deleter_ = [](Storage& s) {
            delete static_cast<T*>(s.large_);
            s.large_ = nullptr;
        };
    }
}
```

5.19.4 Destructor and any_cast

The destructor calls through the type-erased deleter. `any_cast` compares `typeid(T)` against the stored `type_info` and retrieves from the correct storage. `std::launder` is needed when reading from the inline buffer.

Listing 63: Destructor and any_cast

```
~any() {
    if (deleter_) deleter_(storage_);
}

template <typename T>
auto any_cast() const -> const T& {
    if (!type_ || *type_ != typeid(T))
        throw std::bad_cast{};
    if constexpr (is_small_v<T>) {
        return *std::launder(
            reinterpret_cast<const T*>(storage_.small_));
    } else {
        return *static_cast<const T*>(storage_.large_);
    }
}
```

5.20 std::variant

6 Algorithms

6.1 DFS (Depth First Search)

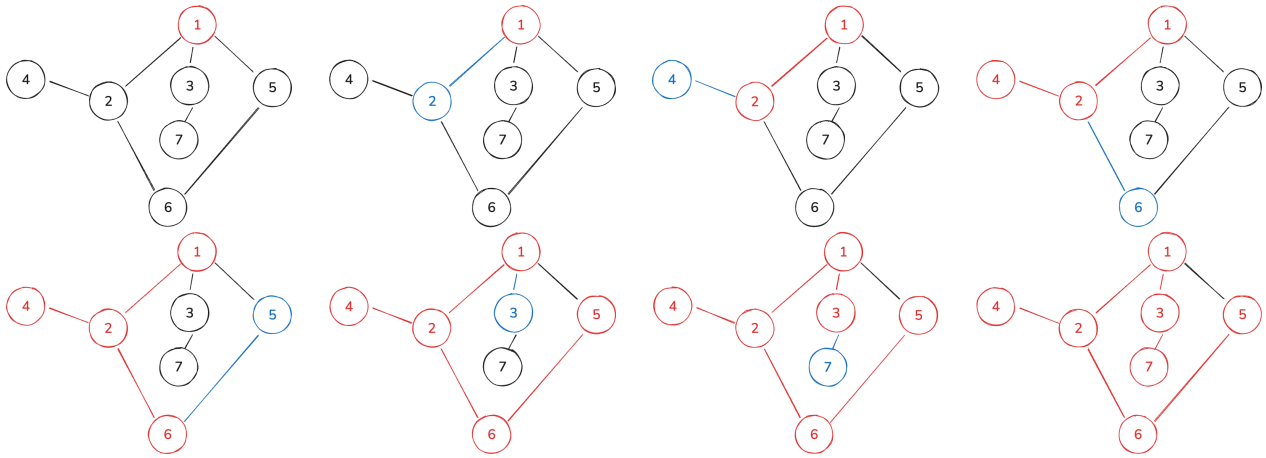


Figure 8: Depth First Search traversal steps (min-neighbor tie rule)

6.2 BFS (Breadth First Search)

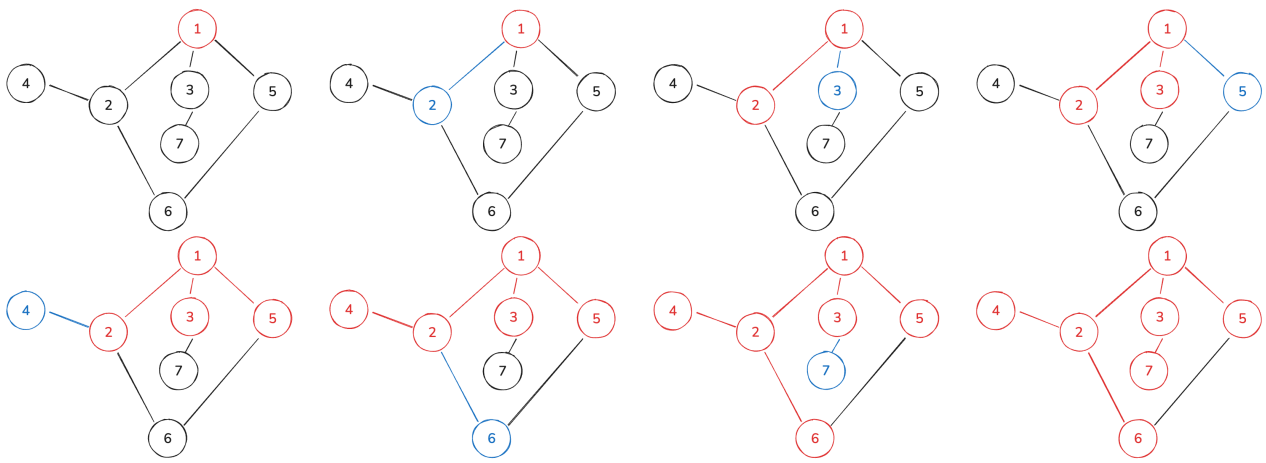


Figure 9: Breadth First Search traversal steps (min-neighbor tie rule)

7 Interview Specifics

7.1 Tricks

7.1.1 Custom Hasher

Suppose you want to use something like the following as a key for a `std::unordered_map`:

```cpp
struct MyStruct {
    int val;
    u32 iter;
    usize idx;

    [[nodiscard]] auto operator==(const MyStruct& other) const noexcept -> bool {
        return (val == other.val) && (iter == other.iter) && (idx == other.idx);
    }
};
```

There is no built-in hash-combiner in the C++ STL, so you have to write it yourself, a good enough hash mixing is the following:

```cpp
struct MyHasher {
    [[nodiscard]] auto operator()(const MyStruct& k) const noexcept -> usize {
        const auto h1 = std::hash<int>{}(k.val);
        const auto h2 = std::hash<u32>{}(k.iter);
        const auto h3 = std::hash<usize>{}(k.idx);
        return h1 ^ (h2 << 1) ^ (h3 << 2);
    }
};
```

then you can use your map with

```cpp
std::unordered_map<MyStruct, int, MyHasher> memo_{};
```

7.1.2 2 Pointer Palindrome iteration

To count up all the palindromes in a string we you can sweep center index `i` left to right and check for odd palindromes

$$\texttt{xs[i - a] = xs[i + a]}$$

and even palindrome

$$\texttt{xs[i - a] = xs[i + 1 + a]}$$

in the following way:

```cpp
const auto n = static_cast<int>(s.size());
auto palindrome_count = 0;
const auto expand_and_update = [&](int left, int right) -> void {
    while(left >= 0 && right < n && s[left] == s[right]) {
        --left;
        ++right;
        ++palindrome_count;
    }
};
for(auto i = 0; i < n; ++i) {
    expand_and_update(i, i); // Odd Palindrome
    expand_and_update(i, i + 1); // Even Palindrome
}
```

7.1.3 Reverse Heap for bounded count

Suppose that you have an array of `n` elements and you want to get the top `k` elements of it. One way of doing this is to create a `MaxHeap` and to push all elements on it and pop k times, but this means you have `n` elements in your heap but only need k.

```cpp
auto top_k_naive(const std::vector<int>& xs, int k) -> std::vector<int> {
    std::priority_queue<int> max_heap{};
    for(const auto x : xs) {
        heap.push(x);
    }
    std::vector<int> out;
    out.reserve(static_cast<usize>(k));
    for(auto i = 0zu; !heap.empty() && i < k; ++i) {
        out.push_back(heap.top());
        heap.pop();
    }
    return out;
}
```

The way to do this is to instead create a `MinHeap` and to iteratively pop elements as we insert them once we have reached our target:

```cpp
auto top_k(const std::vector<int>& xs, int k) -> std::vector<int> {
    std::priority_queue<int, std::vector<int>, std::greater<int>> min_heap{};
    for(const auto x : xs) {
        min_heap.push(x);
        if(min_heap.size() > k) {
            // Pops the smallest element
            min_heap.pop();
        }
    }
    std::vector<int> out;
    out.reserve(static_cast<usize>(k));
    for(auto i = 0zu; !min_heap.empty() && i < k; ++i) {
        out.push_back(min_heap.top());
        min_heap.pop();
    }
    return out;
}
```

Claim. Let $x_1, \ldots, x_n$ be the input values and let $k \geq 1$. Maintain a min-heap H with the rule: push each x_i into H and, if $|H| > k$, pop once. Then after processing all n elements:

1. $|H| = \min(k, n)$,
2. every element in H is among the $\min(k, n)$ largest elements of $\{x_1, \ldots, x_n\}$,
3. the element $\min(H)$ (the heap top) equals the k-th largest element (when $n \geq k$).

Proof (invariant argument). Process the stream left-to-right. For each prefix $P_i := \{x_1, \ldots, x_i\}$ define T_i to be the multiset of the $\min(k, i)$ largest elements of P_i.

We show by induction on i that after processing x_i, the heap content equals T_i as multisets.

Base case ($i = 1$). After pushing x_1, the heap contains $\{x_1\}$ and no pop occurs. Thus $H = \{x_1\} = T_1$.

Induction step. Assume after processing x_{i-1} we have $H = T_{i-1}$. Now insert x_i into H.

Case 1: $i \leq k$. No pop occurs, so $H = T_{i-1} \cup \{x_i\}$. Since $i \leq k$, the set of the $\min(k, i) = i$ largest elements of P_i is just all of P_i, hence $T_i = P_i = T_{i-1} \cup \{x_i\}$, so $H = T_i$.

Case 2: $i > k$. After the push we have $k + 1$ elements, then we pop the minimum element m. So the new heap is

$$H' = \big(T_{i-1} \cup \{x_i\}\big) \setminus \{m\}, \qquad m = \min\big(T_{i-1} \cup \{x_i\}\big).$$

We claim $H' = T_i$.

Indeed, $T_{i-1} \cup \{x_i\}$ consists of $k + 1$ candidates for the top-k of P_i. Removing the smallest element among these $k + 1$ leaves exactly the k largest elements of the union. Equivalently, the only element that can be excluded from the top-k of P_i is the smallest among these $k + 1$ numbers; all remaining elements are $\geq m$ and therefore dominate m. Thus the k-largest multiset of P_i is precisely

$$T_i = \big(T_{i-1} \cup \{x_i\}\big) \setminus \{\min(T_{i-1} \cup \{x_i\})\} = H'.$$

In both cases the invariant holds, so by induction $H = T_n$ after all insertions. Items (1)–(3) follow immediately: when $n \geq k$, H contains exactly the k largest elements, and the minimum of these is the k-th largest overall. $\square$

7.1.4 Power-of-Two Division and Modulo via Bit Tricks

When the divisor is a power of two $s = 2^k$, division and modulo can be replaced by bit shifts and masks:

$$\texttt{x / s} \equiv \texttt{x >> k}, \qquad \texttt{x \% s} \equiv \texttt{x \& (s - 1)}.$$

The key insight is that s has exactly one set bit, so `std::countr_zero(s)` gives $k = \log_2(s)$ by counting trailing zeros. For example `countr_zero(8)` $= 3$ because $8 =$ `0b1000`.

The mask trick (`& (s - 1)`) is used in the `OpenAddressingHashMap` to replace modulo for bucket indexing. The shift trick is useful when you need the quotient, for example to bin values into fixed-width buckets:

Listing 64: Power-of-two histogram bucketing

```cpp
struct Bucket {
    int Start{};
    int End{};
    int Count{};
};

class Histogram
{
public:
    Histogram(usize count, usize size)
        : count_(count), size_(size) {}

    auto Build(const std::vector<int>& latencies) -> std::vector<Bucket>
    {
        const auto count_i = static_cast<int>(count_);
        const auto size_i = static_cast<int>(size_);
        const auto shift = std::countr_zero(size_); // log2(size_)

        std::vector<Bucket> buckets;
        buckets.reserve(count_ + 1);
        for(int i{0}; i < count_i; ++i) {
            buckets.emplace_back(i * size_i, (i + 1) * size_i, 0);
        }
        buckets.emplace_back(
            count_i * size_i, std::numeric_limits<int>::max(), 0);

        for(const auto latency : latencies) {
            // latency / size_ via right shift
            const auto idx = static_cast<usize>(latency) >> shift;
            ++buckets[std::min(idx, count_)].Count;
        }
        return buckets;
    }
private:
    usize count_;
    usize size_; // must be a power of two
};
```

Be careful with signed integers and bitwise shifts; cast to unsigned first. For non-power-of-two divisors the compiler will optimise the division anyway, so these tricks are mainly useful when the power-of-two property is a runtime invariant that the compiler cannot prove.

7.1.5 Debug Macro

You can use macros to quickly print out the names and values of expressions. Note that you should always output to `std::cerr` (I think `std::clog` can also be used but `std::cerr` is safer) as `std::cout` is used to evaluate the result. A compile-time toggle allows disabling all debug output without removing the macro calls.

```cpp
constexpr bool k_do_print{true};

#define DS_DBG(x) if constexpr (k_do_print) \
    std::cerr << #x << " = " << (x) << '\n';
```

```cpp
#define DS_DBG2(x, y) if constexpr (k_do_print) \
    std::cerr << #x << " = " << (x) << ", " << #y << " = " << (y) << '\n';

#define DS_DBG_t1(x) if constexpr (k_do_print) \
    std::cerr << "\t" << #x << " = " << (x) << '\n';
#define DS_DBG_t2(x) if constexpr (k_do_print) \
    std::cerr << "\t\t" << #x << " = " << (x) << '\n';
```

For printing vectors a helper function is useful. The array macro uses the same `k_do_print` toggle.

```cpp
template <typename T>
auto print(const std::vector<T>& xs, std::string_view name) -> void {
    std::cerr << name << " = [";
    for(usize i{0zu}; i < xs.size(); ++i) {
        if(i > 0) {
            std::cerr << ", ";
        }
        std::cerr << xs[i];
    }
    std::cerr << "]\n";
}

#define DS_DBG_ARR(xs) if constexpr (k_do_print) print((xs), #xs);
```

7.1.6 Rolling Average

$$\operatorname{avg}(k) := \frac{1}{k}\sum_{i=1}^{k} a_i.$$

Claim. For all $k \geq 1$,

$$\operatorname{avg}(k+1) = \frac{\operatorname{avg}(k)\,k + a_{k+1}}{k+1}.$$

Proof by induction. Base case.

$$\operatorname{avg}(1) = \frac{1}{1}\sum_{i=1}^{1} a_i = a_1.$$

Induction step. Assume for some $k \geq 1$ that

$$\operatorname{avg}(k) = \frac{1}{k}\sum_{i=1}^{k} a_i.$$

Then

$$\begin{aligned}
\operatorname{avg}(k+1) &= \frac{1}{k+1}\sum_{i=1}^{k+1} a_i && (1)\\
&= \frac{1}{k+1}\left(\sum_{i=1}^{k} a_i + a_{k+1}\right) && (2)\\
&= \frac{1}{k+1}\left(k \cdot \frac{1}{k}\sum_{i=1}^{k} a_i + a_{k+1}\right) && (3)\\
&= \frac{k\operatorname{avg}(k) + a_{k+1}}{k+1}. && (4)
\end{aligned}$$

Thus the identity holds for $k+1$, and therefore for all $k \in \mathbb{N}$ by induction.

Examples.

$$\begin{aligned}
\operatorname{avg}(1) &= a_1, \\
\operatorname{avg}(2) &= \frac{a_1 + a_2}{2} = \frac{\operatorname{avg}(1) \cdot 1 + a_2}{2}, \\
\operatorname{avg}(3) &= \frac{a_1 + a_2 + a_3}{3} = \frac{\operatorname{avg}(2) \cdot 2 + a_3}{3}.
\end{aligned}$$

7.1.7 Partition-based Binary Search (Two Sorted Arrays Median)

Given two sorted sequences A and B with lengths n and m, we often want the *lower median* of the multiset union.

Let $N = n + m$ and define the left-partition size

$$L := \frac{N+1}{2}.$$

We choose counts i and j such that

$$i + j = L, \qquad 0 \le i \le n, \qquad 0 \le j \le m.$$

Define boundary values (with sentinels):

$$A_L = \begin{cases} -\infty, & i = 0, \\ A[i-1], & \text{else}, \end{cases} \qquad A_R = \begin{cases} +\infty, & i = n, \\ A[i], & \text{else}, \end{cases}$$

and analogously B_L, B_R using j. The partition is valid iff

$$A_L \le B_R \quad \text{and} \quad B_L \le A_R.$$

When valid, the lower median is

$$\max(A_L, B_L).$$

We binary search i in the feasible range

$$i \in \big[\max(0, L-m),\ \min(n, L)\big],$$

moving left if $A_L > B_R$ and right otherwise.

7.1.8 Prefix Sums

$$S(k) := \sum_{i=1}^{k} x_i, \qquad S(0) := 0.$$

Claim. For all integers $1 \le i \le k \le n$,

$$\sum_{j=i}^{k} x_j = S(k) - S(i-1).$$

Proof. By definition,

$$S(k) = \sum_{j=1}^{k} x_j \quad \text{and} \quad S(i-1) = \sum_{j=1}^{i-1} x_j.$$

Subtracting yields

$$S(k) - S(i-1) = \sum_{j=1}^{k} x_j - \sum_{j=1}^{i-1} x_j \tag{5}$$

$$= \sum_{j=i}^{k} x_j. \tag{6}$$

Examples.

$$\begin{aligned}\sum_{j=3}^{5} x_j &= x_3 + x_4 + x_5 \\ &= (x_1 + x_2 + x_3 + x_4 + x_5) - (x_1 + x_2) \\ &= S(5) - S(2).\end{aligned}$$

Thus, once the cumulative sum array $S(k)$ is known, any subarray sum can be computed in constant time via

$$\sum_{j=i}^{k} x_j = S(k) - S(i-1).$$

7.2 Problems

7.2.1 Longest common subsequence

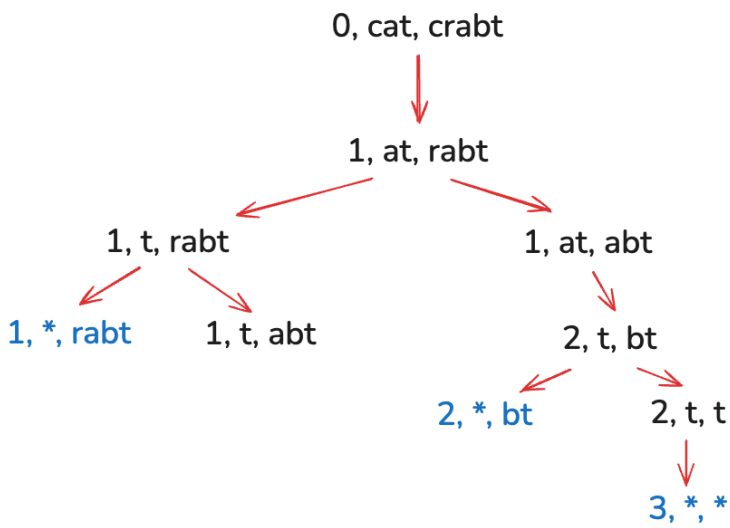


Figure 10: Longest Common Subsequence

7.2.2 Longest increasing subsequence

Given an array of numbers you want to find the length of the longest increasing subsequence. The fancy solution (called Patience Sorting) for that is to use binary search

```
auto longest_increasing_subsequence(const std::vector<int>& xs) -> int {
    std::vector<int> tails(xs.size());
    for(const auto x : xs) {
        auto it = std::lower_bound(tails.begin(), tails.end(), x);

        if(it == tails.end()) {
            tails.push_back(x);
        } else {
            *it = x;
        }
    }
    return static_cast<int>(tails.size());
}
```

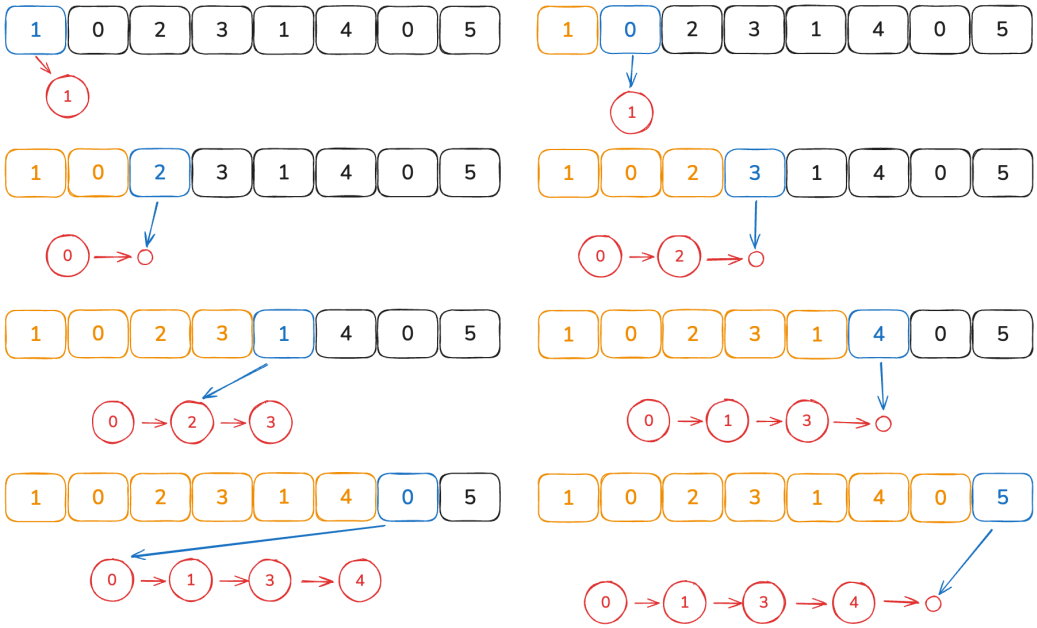


Figure 11: Patience Sort

7.2.3 Unique Paths

This is basically THE intro DP / memoization problem. Suppose you start at `(0, 0)` and you can only go to the right or to the bottom. How many unique paths are there to get to `(m, n)`?

Idea is to memoize on every position, the value of very position is simply the sum of the value of going right once and of going going down once, this causes a lot of overcounting, with memoization we avoid this overcounting.

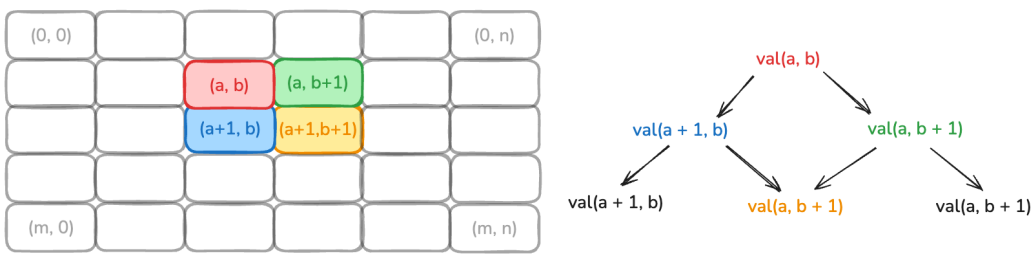


Figure 12: Unique Paths Memoization

```cpp
constexpr auto k_not_init = -1;

class Solution {
    auto uniques_path(int m, int n) -> int {
        m_ = m;
        n_ = n;
        memo_.assign(m * n, k_not_init);

        return helper(0, 0);
    }

private:
    std::vector<int> memo_;
    int m_;
    int n_;

    auto get(int m, int n) -> usize {
        return m * n_ + n;
    }
```

```

    auto helper(int m, int n) -> int {
        if(m == m_ - 1 && n == n_ - 1) return 1;
        if(m >= m_ || n >= n_) return 0;

        const auto idx = get(m, n);
        if(memo_[idx] == k_not_init) {
            memo_[idx] = helper(m + 1, n) + helper(m, n + 1);
        }
        return memo_[idx];
    }
};
```

7.2.4 Merge sorted linked lists

Given two (ascendingly) sorted linked lists `a`, `b` we want to merge `b` into `a` such that a is still sorted. The main tricky part is to realise that adding a dummy node makes the merging much simpler.

```
auto merge2(ListNode* a, ListNode* b) -> ListNode* {
    ListNode dummy();
    ListNode* tail = &dummy;

    while(a && b) {
        if(a->val <= b->val) {
            tail->next = a;
            a = a->next;
        } else {
            tail->next = b;
            b = b->next;
        }
        tail = tail->next;
    }
    tail->next = a ? a : b;
    return dummy.next;
}
```

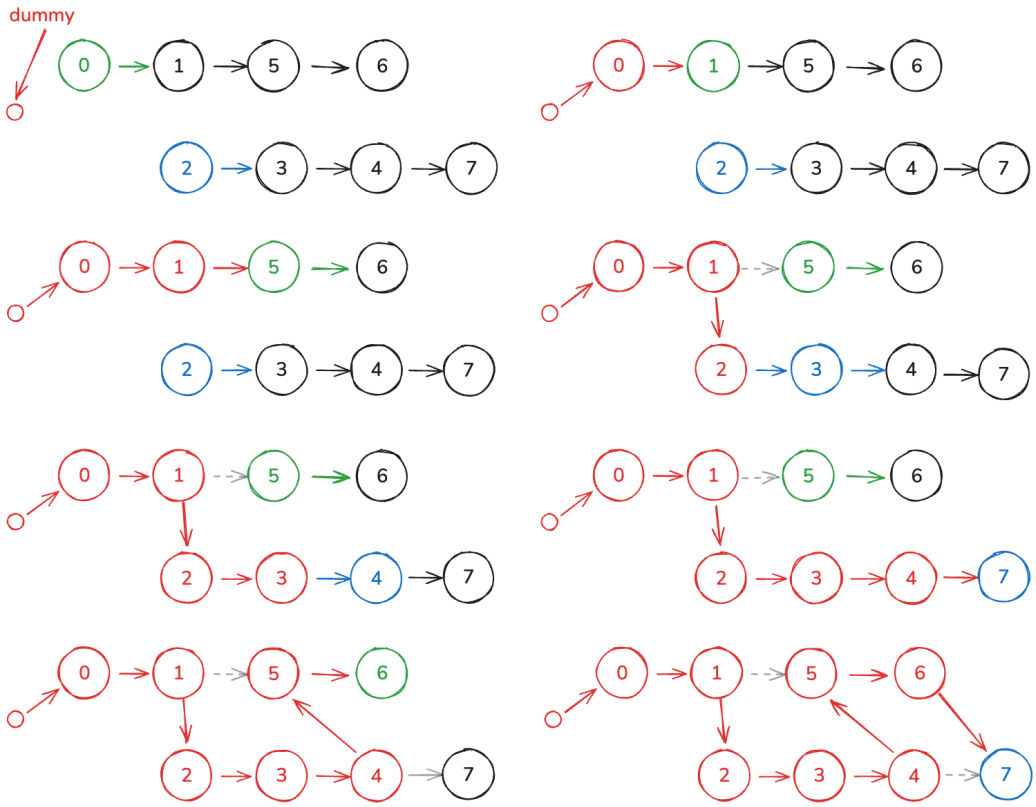


Figure 13: Merge two sorted linked lists

This can then easily be extended to merging arbitrarily many linked lists:

```
auto mergeK(std::vector<ListNode*> lists) -> ListNode* {
    if(lists.empty()) {
        return nullptr;
    }
    if(lists.size() == 1) {
        return lists[0];
    }
    auto curr = lists[0];
    for(auto i = 1zu; i < lists.size(); ++i) {
        curr = merge2(curr, lists[i]);
    }
    return curr;
}
```

If the lists don't alias then this is embarrassingly parallel, can just do a type of tree reduce on the individual combinations.

7.2.5 Rotting Fruit

```
auto orangesRotting(vector<vector<int>>& grid) -> int {
    const auto n = static_cast<int>(grid.size());
    const auto m = static_cast<int>(grid[0].size());

    auto n_fresh = 0;
    std::queue<Pos> rotten_current_step{};
    std::queue<Pos> rotten_next_step{};
    for(auto y = 0; y < n; ++y) {
        for(auto x = 0; x < m; ++x) {
            const auto cell = grid[y][x];
            if(cell == k_fresh) {
                ++n_fresh;
            } else if (cell == k_rotten) {
```

```
                rotten_current_step.push({y, x});
            }
        }
    }

    const auto process_neighbor = [&](int y, int x) {
        if(y < 0 || y >= n || x < 0 || x >= m) return;
        if(grid[y][x] != k_fresh) return;
        grid[y][x] = k_rotten;
        --n_fresh;
        rotten_next_step.push({y, x});
    };
    auto time = 0;
    while(true) {
        while(!rotten_current_step.empty()) {
            const auto [y, x] = rotten_current_step.front();
            rotten_current_step.pop();

            process_neighbor(y, x + 1);
            process_neighbor(y, x - 1);
            process_neighbor(y + 1, x);
            process_neighbor(y - 1, x);
        }
        if(rotten_next_step.empty()) {
            break;
        }
        std::swap(rotten_current_step, rotten_next_step);
        ++time;
    }
    if(n_fresh > 0) {
        return k_fruit_remain_at_end;
    }
    return time;
}
```

7.3 Clone Graph

Suppose you are given a graph node `Node*`, adjacency is encoded by a per-element `std::vector<Node*>`. We are interested in making a deep copy of that graph.

Core idea is to do a `DFS` (or `BFS`) through the graph to touch all nodes, store the "new-to-old" `Node` relationship in a `std::unordered_map`.

```
using OldNode = Node;
using NewNode = Node;

auto graph_deep_copy(OldNode* node) -> Node* {
    if(!node) return nullptr;

    std::unordered_map<OldNode*, NewNode*> old_to_new{};
    std::stack<OldNode*> to_visit{};

    old_to_new[node] = new NewNode(node->val);
    to_visit.push(node);
    while(!to_visit.empty()) {
        OldNode* curr = to_visit.top();
        to_visit.pop();

        NewNode* curr_cpy = old_to_new[curr];
        for(OldNode* nb : curr->neighbors) {
            if(!nb) continue;
            if(const auto it = old_to_new.find(nb); it == old_to_new.end()) {
                old_to_new[nb] = new NewNode(nb->val);
                to_visit.push(nb);
```

```
            }
            curr_cpy->neighbors.push_back(old_to_new[nb]);
        }
    }
    return old_to_new[node];
}
```

7.3.1 Maximum Subarray

Let `nums` be an array of integers, a subarray is a contiguous non-empty sequence of elements in that array. We are interested in finding the largest subarray.

The solution to this is known as Kadane's algorithm, we greedily decide in each step if we should start a new subarray or extend the current one. There is never a reason to backtrack.

```
auto best_sum = std::numeric_limits<int>::lowest();
auto current_sum = 0;
for(const auto num : nums) {
    current_sum = std::max(
        current_sum + num, // Extend current subarray
        num // Make a new subarray
    );
    best_sum = std::max(best_sum, current_sum);
}
return best_sum;
```

7.3.2 Maximum Product Subarray

This is a variation on the "Maximum Subarray" problem where we consider products. This case is a bit more tricky for two reasons:

- Products with 0 become 0
- We must also track rolling min because product with negative number makes larger number MORE negative than smaller number (assuming both are positive)

```
auto max_product_subarray(const std::vector<int>& nums) -> int {
    auto max_prod = nums[0];
    auto min_prod = nums[0];
    auto best_prod = nums[0];

    for(auto i = 1zu; i < nums.size(); ++i) {
        const auto x = nums[i];

        if(x < 0) std::swap(max_prod, min_prod);

        max_prod = std::max(max_prod * x, x);
        min_prod = std::min(min_prod * x, x);

        best_prod = std::max(best_prod, max_prod);
    }
    return best_prod;
}
```

7.3.3 Median of Two Circularly Sorted Logs

This uses Partition-Based Binary Search with some pivoted indexing.

Suppose you are given two vectors `A`, `B` which are rotations of sorted lists. This means there exists `k_A`, `k_B` such that

`[A[k_A], A[k_A + 1], A[k_A + 2], ...]`, `[B[k_B], B[k_B + 1], A[k_B + 2], ...]`

are sorted. The rotation offset (which I call pivot) can then be computed by using binary search

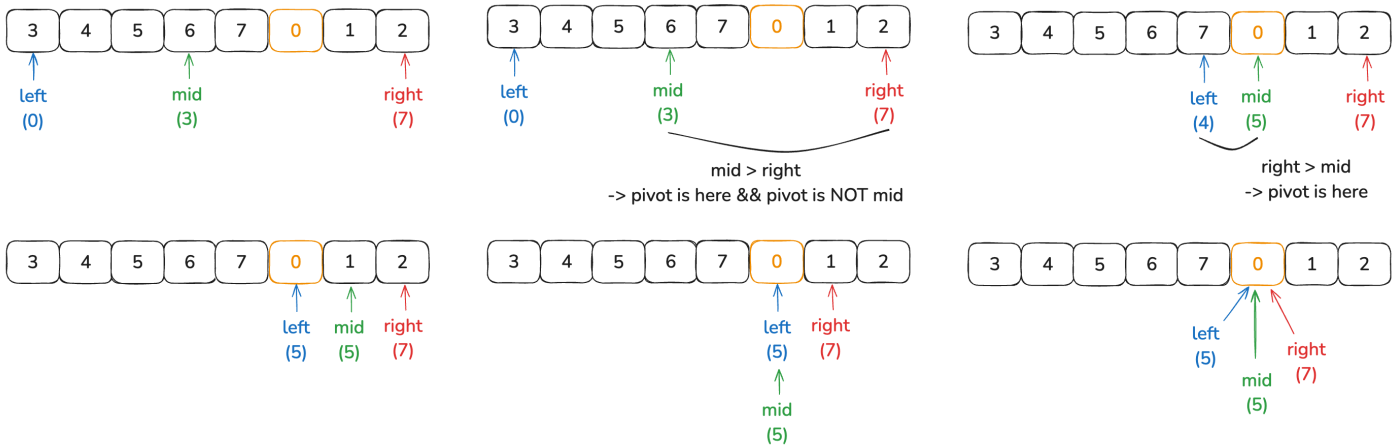


Figure 14: Pivot + partition-binary-search steps (circularly sorted arrays)

```cpp
template <typename T>
auto compute_pivot(std::span<const T> xs) -> usize {
    if (xs.empty()) { return 0zu; }

    auto left = 0zu;
    auto right = xs.size() - 1zu;
    while (left < right) {
        const usize mid = std::midpoint(left, right);

        if (xs[mid] < xs[right]) {
            // Pivot in RHS
            right = mid;
        }
        else if (xs[mid] > xs[right]) {
            // Pivot in LHS
            left = mid + 1zu;
        }
        else {
            assert(xs[mid] == xs[right]);
            --right;
        }
    }
    return left;
}
```

With the pivot we can then shift and get an element of the un-pivoted list:

```cpp
template<typename T, typename IndexT>
auto get_rotated(std::span<const T> xs, IndexT shift, IndexT idx) {
    return xs[static_cast<usize>(shift + idx) % xs.size()];
}
```

As a sidenote, this kind of indexing is expensive for efficient data structures like ring buffers, there we require the size to be a multiple of 2 and we just mask it off directly, but this can't be assumed here.

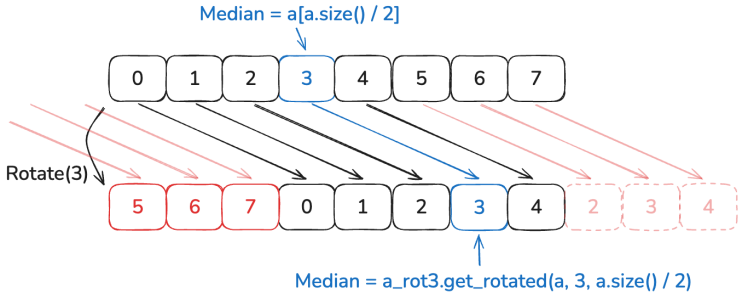


Figure 15: Median of Rotation

If one of the two lists is empty then this problem reduces to finding the pivot of the rotation of a sorted list, which by definition just means getting the middle element in the un-rotated list:

```
const auto pivot_A = compute_pivot(A);
if(B.empty()) {
    assert(!A.empty());
    return get_rotated(A, pivot_A, (A.size() - 1zu) / 2zu);
}
```

The general case first computes how my elements we want to have to the left of the median, which is

`num_left = (A.size() + B.size() + 1) / 2`

compute bounds for what ranges in A this could be satisfied with and then doing binary search to find the desired result.

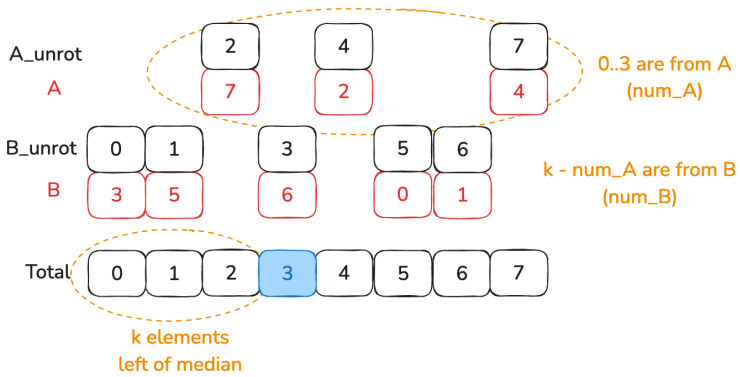


Figure 16: Visualisation of Bounds

We are interested in having `num_left = num_A + num_B` elements in total where `num_{A, B}` are the number of elements on the left partition. We exit when we have a valid partition, which is categorized by us preserving the ordering (left most element in A is left of right most element of B and v.v.).

```
const auto num_A_min = std::max(0, num_left - B.size());
const auto num_A_max = A.size();
assert(num_A_min <= num_A_max);
while(true) {
    const auto num_A = (num_A_min + num_A_max + 1) / 2;
    const auto num_B = num_left - num_A;

    constexpr auto long_min = std::numeric_limits<long>::min();
    constexpr auto long_max = std::numeric_limits<long>::max();

    const auto A_left  = (num_A == 0)        ? long_min : get_rotated(A, pivot_A, num_A - 1);
    const auto A_right = (num_A == A.size()) ? long_max : get_rotated(A, pivot_A, num_A);

    const auto B_left  = (num_B == 0)        ? long_min : get_rotated(B, pivot_B, num_B - 1);
    const auto B_right = (num_B == B.size()) ? long_max : get_rotated(B, pivot_B, num_B);
```

```

    if(A_left <= B_right && B_left <= A_right) { // Partition is Valid
        return std::max(A_left, B_left);
    }

    if(A_left > B_right) {
        num_A_max = num_A - 1;
    } else {
        assert(B_left > A_right);
        num_A_min = num_A + 1;
    }
}
```

7.3.4 Lazy Deletion Stack

This uses the Lazy Delete / Tombstone pattern to allow efficiently deleting large parts of a stack to implement `remove_lower` and `remove_upper`, which cut out (potentially) large parts of the stack. Aside from tombstoning a `std::map` is used (as opposed to `std::unordered_map`) to avoid having to do a linear sweep over the whole stack when deleting objects.

```
class LazyDeleteStack {
public:
    auto push(int value) -> void {
        const usize idx = data_.size();
        data_.push_back({value, false});
        value_to_idx_[value].push_back(idx);
    }

    auto pop() -> void {
        while (!data_.empty()) {
            const auto top = data_.back();

            const auto entry = data_.back();
            data_.pop_back();
            value_to_idx_[entry.value].pop_back();

            if (!top.is_removed) {
                // break on first alive object deleted
                break;
            }
        }
    }

    auto remove_lower(int value) -> void {
        auto it = value_to_idx_.begin();
        const auto end = value_to_idx_.lower_bound(value);
        while (it != end) {
            for (auto idx : it->second) {
                data_[idx].is_removed = true;
            }
            ++it;
        }
    }

    auto remove_upper(int value) -> void {
        auto it = value_to_idx_.upper_bound(value);
        while (it != value_to_idx_.end()) {
            for (auto &idx : it->second) {
                std::println("remove_upper inner");
                data_[idx].is_removed = true;
            }
            ++it;
        }
    }

```

```
    auto print() const -> void {
        for (auto i = data_.size(); i-- > 0;) {
            const auto entry = data_[i];
            if (!entry.is_removed) {
                std::println("<{:3}>", entry.value);
            }
        }
    }

private:
    std::vector<StackEntry> data_{};
    std::map<int, std::vector<usize>> value_to_idx_{};
};
```

7.3.5 Subarrays with Given Sum and Bounded Maximum

Suppose we are given an array `nums` with `n` elements and we are interested in counting the number of contiguous subarrays which sum to some `k` and whose elements are at most `M`.

First we note that whenever we see a value `x > M` we have a cut, so we can see this problem as summing up the number of contiguous subarrays per block, where blocks are separated by too large values. For example suppose

`nums = [-1, 2, 1, 7, -1, 5, 2, 1, 2, -7]`,

`k = 2` and `M = 3`, then

```
[ -1, 2,  1,  7, -1, 5, 2, 1, 2, -7]
              ^ {-1, 2, 1}
                    ^ {-1}
                                ^ {2, 1, 2, -7}
```

and `F(nums, k, M) = f(-1, 2, 1, k) + f(-1, k) + f(2, 1, 2, -7, k)` where `F` denotes the main entry point and `f` the counts per block.

Let `pref(x)` for an array `x = {x1, x2, x3, ...}` be defined as the cumulative sum

`pref(x) = {x1, x1 + x2, x1 + x2 + x3, ...}`.

We can then efficiently evaluate the total sum of a subarray `[xi, ..., xk]` by computing

`pref(x)[k] - pref(x)[i - 1]`

(of course you cache and don't recompute it every time). The big speedup improvement comes from inserting target values into a hash map and summing up the counts while having a single running total instead of storing the prefix array.

```
++counts[pref - k];
out += counts[pref];
```

To avoid unnecessary insertions by using `[]` we use `.find` and obtain the following per-block logic:

```
if(auto it = counts.find(pref - k); it != counts.end()) {
    out += it->second;
}
++counts[pref];
```

A block ends when a value is `> M` so we get the following total solution

```
auto out = 0ll;
for(auto i = 0zu; i < nums.size(); ++i) {
    const auto num = nums[i];
    if(num > M) {
        counts.clear();
        pref = 0ll;
        continue;
    }
    if(auto it = counts.find(pref - k); it != counts.end()) {
```

```
        out += it->second;
    }
    ++counts[pref];
}
return out;
```

7.4 Maximize Profit with Task Deadlines and Multiple Servers

Suppose we have `n` tasks and `m` workers. Each worker can process one task in a single timestep. Tasks have a profit and a deadline, if a task is completed before its deadline then we get that profit. The problem is to figure out what the optimal assignment of workers is to maximise the total profit.

Because each task has the same duration the optimal strategy is a rolling greedy strategy. We first collect the lists of deadlines and profits into combined Task objects (alternatively one could do permutation on `std::iota` and use that as a permutation map, but this would only make sense for significantly larger lists).

```
template <typename DeadlineT, typename ProfitT>
struct TaskT {
    DeadlineT deadline;
    ProfitT profit;
};
template <typename DeadlineT, typename ProfitT>
auto collect_tasks(const std::vector<DeadlineT>& deadlines, const std::vector<ProfitT>& profits) ->
    std::vector<TaskT<DeadlineT, ProfitT>> {
    assert(deadlines.size() == profits.size());
    std::vector<TaskT<DeadlineT, ProfitT>> out;
    out.reserve(deadlines.size());
    for(auto i = 0zu; i < deadlines.size(); ++i) {
        out.emplace_back(deadlines[i], profits[i]);
    }
    return out;
}
using Task = TaskT<int, int>;
```

These tasks must be sorted by their deadlines (we can't use ranges as HackerRank is stuck on C++20)

```
std::vector<Task> tasks = collect_tasks(deadlines, profits);
std::sort(
    tasks.begin(),
    tasks.end(),
    [](const Task& a, const Task& b) -> bool {
        return a.deadline < b.deadline;
    });
```

To efficiently solve this problem we need to be able to quickly evict tasks if they are not worth doing. This is ideally solved by using a `MinHeap`:

```
using MinHeap = std::priority_queue<int, std::vector<int>, std::greater<int>>;
// using MaxHeap = std::priority_queue<int, std::vector<int>, std::less<int>>;
using MaxHeap = std::priority_queue<int>;
```

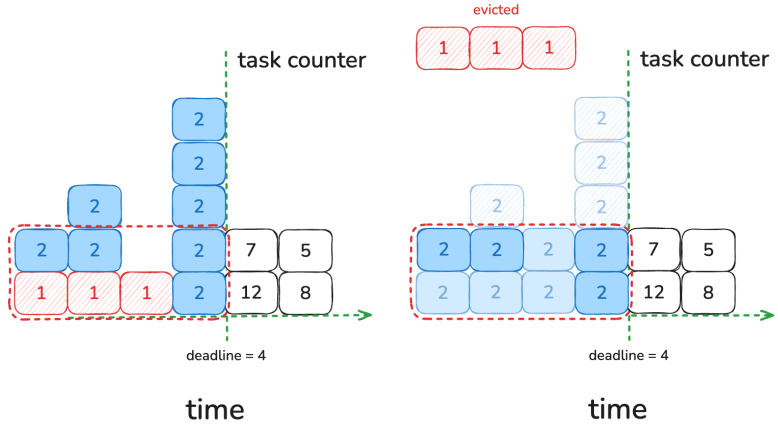


Figure 17: Maximize profit with task deadlines and multiple servers

What remains is to iterate through all tasks and check that current rolling window if a task should be evicted due to capacity. We track the total profits and greedily try to improve it.

```
MinHeap counts{};
auto sum = 0l;
for(auto [deadline, profit] : tasks) {
    counts.push(profit);
    sum += profit;

    const auto capacity = static_cast<i64>(num_workers) * static_cast<i64>(d);
    if(counts.size() >= static_cast<usize>(capacity)) {
        // Evict worst task
        sum -= counts.top();
        counts.pop();
    }
}
```

8 Software Engineering

8.1 Testing, Unit Tests, TDD (Test Driven Development)

Good book on testing is 15.

The most famous testing frameworks for C++ are Catch2 and Google Test (also known as GTest).

Attributes of tests to focus on:

Falsifiable	Tests set up falsifiable hypothesis. If your "Test Oracle" is your own code then your test is not falsifiable, as you just prove "The code we wrote is the code we wrote". That is NOT a unit test but an acceptance test (those are important for opaque and/or legacy code you don't understand).
Repeatable	You get the same answer every time.
Replicable	Your colleagues get the same answer as you do.
Accuracy	Measurements are "right".
Precision	Measurements are "informative".

8.2 Design Patterns

8.2.1 Visitor Pattern

In C++ this can be implemented efficiently (no runtime overhead) by using `std::variant` and `std::visit`.

8.2.2 Strategy Pattern

8.2.3 Factory Method

A factory function encapsulates object creation behind an interface, returning a base-class pointer (`std::unique_ptr` or `std::shared_ptr`). The caller doesn't know or care which concrete type is created. Useful for configuration-driven instantiation (e.g. selecting a logger or parser based on runtime input).

Listing 65: Factory method with singleton caching

```cpp
struct ILogger {
    virtual ~ILogger() = default;
    virtual void log(const std::string&) = 0;
};

struct ConsoleLogger : ILogger {
    void log(const std::string& msg) override {
        std::cout << "LOG: " << msg << std::endl;
    }
};

struct NullLogger : ILogger {
    void log(const std::string&) override {}  // do nothing
};

enum class LoggerType { None, Console };

auto GetLogger(std::optional<LoggerType> type)
    -> std::shared_ptr<ILogger>
{
    // static map: one allocation per type (singleton)
    const static std::unordered_map<
        LoggerType, std::shared_ptr<ILogger>> loggers{
        {LoggerType::None, std::make_shared<NullLogger>()},
        {LoggerType::Console, std::make_shared<ConsoleLogger>()},
    };

    if (!type) return loggers.at(LoggerType::None);
    return loggers.at(*type);
}
```

Key points: the `static` map ensures each logger type is constructed exactly once. `NullLogger` is the Null Object pattern — a do-nothing implementation that avoids null checks at every call site.

8.2.4 Abstract Factory

When a family of related objects must be created together (e.g. an exchange needs both an order sender and a market data feed), the Abstract Factory groups creation methods in a single interface. Each concrete factory produces a consistent family.

Listing 66: Abstract Factory: exchange components

```cpp
struct OrderSender {
    virtual ~OrderSender() = default;
    virtual std::string Name() const = 0;
};
struct MarketDataFeed {
    virtual ~MarketDataFeed() = default;
    virtual std::string Name() const = 0;
};

struct ExchangeFactory {
    virtual ~ExchangeFactory() = default;
    virtual std::unique_ptr<OrderSender>
        CreateOrderSender() const = 0;
    virtual std::unique_ptr<MarketDataFeed>
        CreateMarketDataFeed() const = 0;
};

struct NasdaqOrderSender : OrderSender {
    std::string Name() const override {
        return "NasdaqOrderSender";
    }
};
struct NasdaqMarketDataFeed : MarketDataFeed {
    std::string Name() const override {
        return "NasdaqMarketDataFeed";
    }
};

struct NasdaqFactory : ExchangeFactory {
    std::unique_ptr<OrderSender>
        CreateOrderSender() const override {
        return std::make_unique<NasdaqOrderSender>();
    }
    std::unique_ptr<MarketDataFeed>
        CreateMarketDataFeed() const override {
        return std::make_unique<NasdaqMarketDataFeed>();
    }
};
// NyseFactory follows the same pattern
```

The caller programs against `ExchangeFactory` and gets a consistent set of components without knowing the concrete types. Adding a new exchange means adding one factory class — no changes to existing code (Open-Closed Principle).

8.2.5 CRTP (Curiously Recurring Template Pattern) Design Pattern

CRTP achieves static (compile-time) polymorphism by having a base class templated on the derived class. The base can call derived-class methods via `static_cast<const Derived*>(this)`, resolving the call at compile time with zero runtime overhead (no vtable, no indirection). Compare with the equivalent runtime polymorphism using `virtual`:

Listing 67: Runtime polymorphism (virtual)

```cpp
struct Elf {
    virtual std::string_view Name() const = 0;
};
struct ArcherElf : Elf {
    std::string_view Name() const override { return "Legolas"; }
};
```

Listing 68: CRTP: compile-time polymorphism

```cpp
template <typename Derived>
struct Elf {
    [[nodiscard]] auto Name() const -> std::string_view {
        return static_cast<const Derived*>(this)->Name();
    }
};
struct ArcherElf : Elf<ArcherElf> {
    std::string_view Name() const { return "Legolas"; }
};
```

The trade-off is that CRTP requires the concrete type to be known at compile time (no heterogeneous containers of `Elf*`), but eliminates the vtable pointer per object and enables inlining.

8.2.6 Type Erasure Design Pattern

9 Databases

9.1 MySQL

9.2 MongoDB

10 Computer Architecture, HPC (High Performance Computing)

10.1 Memory Hierarchy and Cache

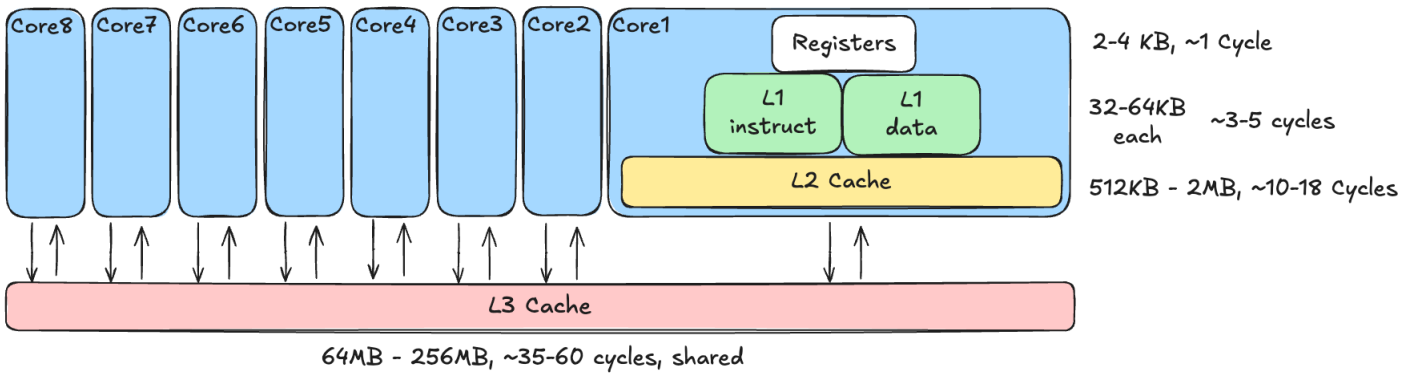


Figure 18: CPU cache hierarchy (registers, L1/L2/L3, main memory)

10.1.1 Cache Lines and Cache Locality

A `Word` is the width of a standard register, on modern machines this is usually `8 Byte`. The architecture defines the size of a pointer, usually it's either `4 Byte` (on 32 bit architecture) or `8 Byte` (on 64 bit architecture). A `Cache Line` is the smallest contiguous chunk of memory that the CPU can load at once. Usually it's the size of `8 Word`. We will assume that

$$\texttt{CacheLine = 8 Word = 8 * 8 Byte = 64 Byte}.$$

10.1.2 Spatial vs Temporal Locality

Spatial Locality means that if you access some element then you probably will access nearby elements as well, this is referred to as spatial locality. For example if you pull a variable into cache on a cache miss you pull in an entire cacheline, if you iterate through a large number of elements sequentially (potentially strided) the CPU will see this and pre-fetch memory to be used when it's needed (pipelining).

```
for(auto i = 0zu; i < vec.size(); ++i) {
    // vec[i + 1] is already ready to be processed before this operation is done
    vec[i] += 1;
}
```

Temporal Locality means that if you access some element then you will probably access it again soon, so it makes sense to cache it.

```
auto x = 1;
for(auto i = 0zu; i < vec.size(); ++i) {
    x += 2; // x remains in L1 cache, doesn't get evicted.
    vec[i] += x;
}
```

10.1.3 False Sharing

False sharing is a phenomenon that appears when you have two different threads write to elements on the same cache line. When thread a writes while thread b uses another value on the same cache line the cache line gets invalidated and so b has a cache miss and needs to pull the value into cache again despite it not having changed. To avoid this make sure that if two threads access memory often that they are not on the same cache line (easiest solution is to just pad to the next cache line, you can do that using the `std::hardware_destructive_interference_size` constant.

```
constexpr usize k_cacheline_size{std::hardware_destructive_interference_size};

struct Packed {
```

```
    int a{};
    int b{};
};

struct Separated {
    alignas(k_cacheline_size) int a{};
    alignas(k_cacheline_size) int b{};
};

auto func_a(Packed& p) -> void {
    p.a = p.a + 1; // invalidates cache line that Packed lives on
}

auto func_b(Packed& p) -> void {
    p.b = p.b + 1; // invalidates cache line that Packed lives on
}

auto func_a_separated(Separated& p) -> void {
    p.a = p.a + 1; // Doesn't invalidate cacheline
}

auto func_b_separated(Separated& p) -> void {
    p.b = p.b + 1; // Doesn't invalidate cacheline
}
```

10.2 Memory Layout and Alignment

10.2.1 Alignment vs Size

10.2.2 Tight Packing vs Padding

10.2.3 AoS vs SoA vs AoSoA

Suppose you have the following representation of a physics owning entity represented as a fat struct

```
struct Transform {
    Pos3 pos;
    Dir3 scale;
    Quaternion orientation;
};
struct Entity {
    u32 id{};
    Color3 color{};
    u32 visual_mask{};
    u32 hit_mask{};
    std::string name{};
    Transform transform{};
    std::unique_ptr<RigidBody> body{};
};
```

If we store our entities as

```
std::vector<Entity> entities{};
```

we have an (A)rray (o)f (S)tructs.

If we now want to update the position of all entites (for example by shifting them all, or syncing the transform with the underlying physics representation) then we'd have to load at least one `CacheLine` per entity which is a lot of wasted loading if we are only interested in the transform, for example

```
sizeof(Transform) == sizeof(Pos3) + sizeof(Dir3) + sizeof(Quaternion) == 3 * 4 + 3 * 4 + 4 * 4 == 40
```

(assuming everything consists of 32 bit floats) while

```
sizeof(Entity) == sizeof(u32) + sizeof(Color3) + sizeof(u32) + sizeof(u32)
                  + sizeof(std::string) + sizeof(Transform) + sizeof(std::unique_ptr)
```

```
                == 4 + 3 * 4 + 4 + 4 + 3 * 8 + 40 + 8
                == 96
```

Assuming things are aligned well we don't have to pull in 2 `CacheLine` but we still pull 24 Bytes of memory too much.

The code for shifting everything by `{1.0f, 1.0f, 1.0f}` is

```
const Vec3 shift{1.0f, 1.0f, 1.0f};
for(auto& entity : entities) {
    entity.transform += shift;
}
```

We instead can store the individual components contiguously in memory

```
TransformSOA {
    f32* pos_xs;
    f32* pos_ys;
    f32* pos_zs;
    f32* scale_xs;
    f32* scale_ys;
    f32* scale_zs;
    Quaternion* orientations;
}
```

and now we can load in three `CacheLine` to update 8 elements at a time, so we have 3 cache misses per 16 updates in the worst case. The pointers can for example store to a ArenaAllocator or some other contiguous storage like `std::vector` or `std::pmr::vector`). The update code becomes (assuming number of entities is divisible by 8 to avoid having to deal with boundary behavior)

```
const auto shift_x = 1.0f;
const auto shift_y = 1.0f;
const auto shift_z = 1.0f;
for(auto i = 0zu; i < n_entities; ++i) {
    pos_xs[i] += shift_x;
    pos_ys[i] += shift_y;
    pos_zs[i] += shift_z;
}
```

A higher overhead but more SIMD aware storage method would be to use AoSoA, where we store arrays of blocks (here with element arrays of size 4 as I'm on NEON, can increase for AVX and AVX512):

```
struct TransformBlock {
    std::array<f32, 8> pos_x;
    std::array<f32, 8> pos_y;
    std::array<f32, 8> pos_z;
    /*...*/
};
```

10.2.4 SIMD Alignment Requirements

10.2.5 Practical Trade-offs (Bandwidth vs Compute)

10.3 CPU Microarchitecture

10.3.1 Pipelining

10.3.2 Branch Prediction and Speculative Execution

10.3.3 Out-of-Order Execution

10.4 GPU Architecture

10.4.1 Streaming Multiprocessor (SM) and Warp Execution

The SM is the fundamental compute unit of a GPU. Each SM is partitioned into four *processing blocks* (sub-partitions), each containing:

- One **warp scheduler** with one dispatch unit (one instruction per cycle from the selected warp).

- A slice of the **register file** (16 384 32-bit registers per block, 65 536 per SM on A100).
- **Execution units:** 16 INT32 cores, 16 FP32 cores, 8 FP64 cores, and one tensor core.
- **8 LD/ST units** and **4 SFUs** (Special Function Units for `sin`, `exp`, rsqrt, etc.).

The four processing blocks share the SM's L1 data cache and configurable shared memory (up to 164 kB on A100). Threads are grouped into *warps* of 32, and each warp scheduler selects one resident warp per cycle. Because the four schedulers operate independently, an SM can have four instructions from four different warps in flight simultaneously; this is the primary mechanism for hiding memory and instruction latency.

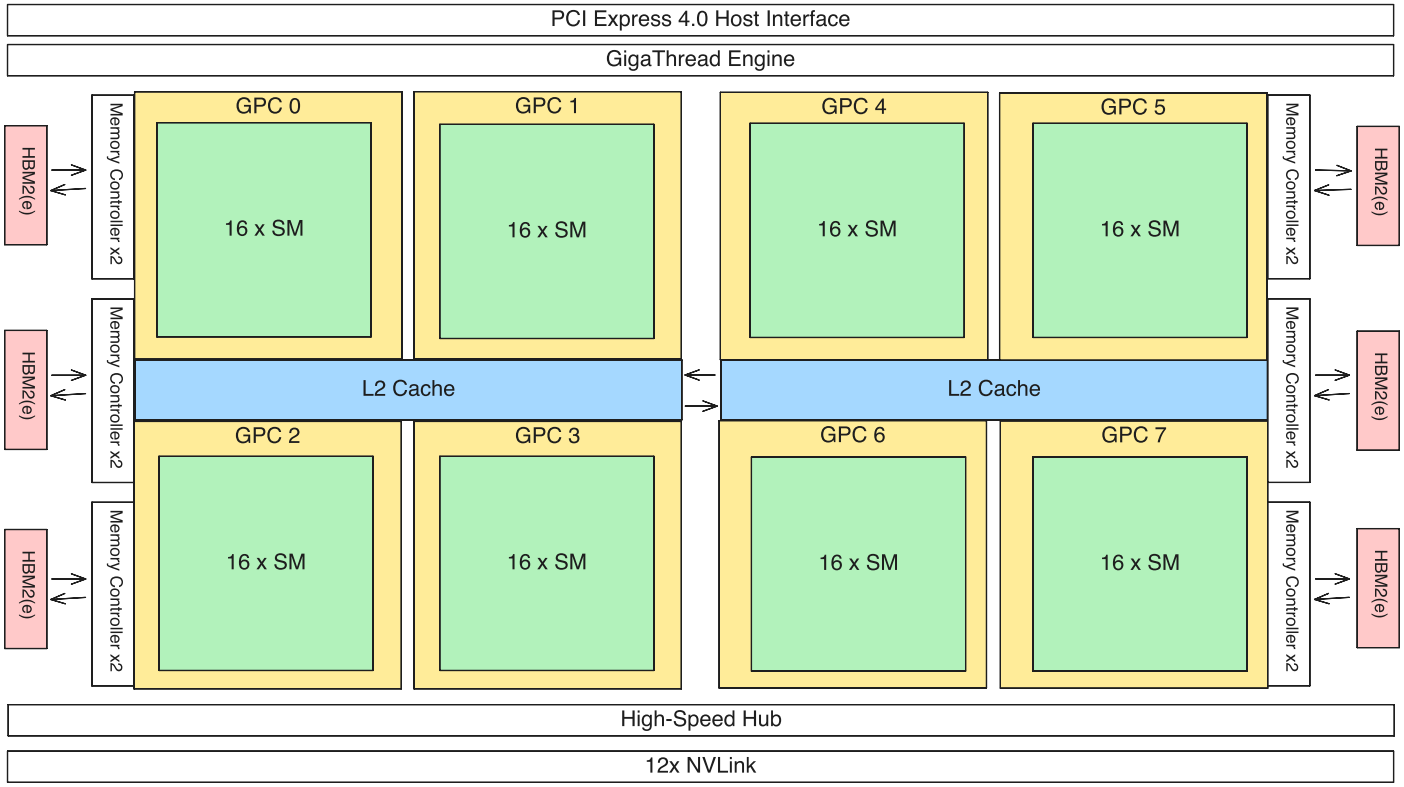


Figure 19: GA100 floorplan: GPCs, L2 cache fabric, memory controllers, and NVLink/PCIe interfaces.

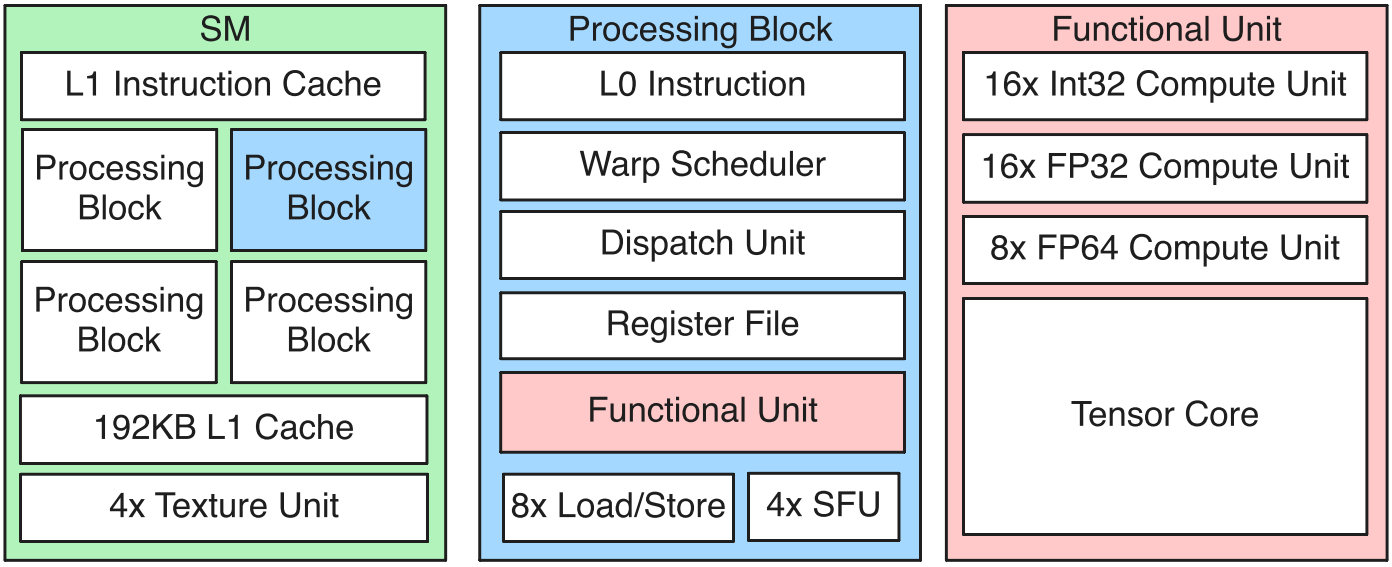


Figure 20: A100 SM internals. **Left:** four processing blocks sharing L1/shared memory. **Centre:** each block's warp scheduler, register file, execution units, LD/ST, SFUs. **Right:** per-block execution units (16 INT32, 16 FP32, 8 FP64, 1 tensor core).

10.4.2 GPU Memory Hierarchy

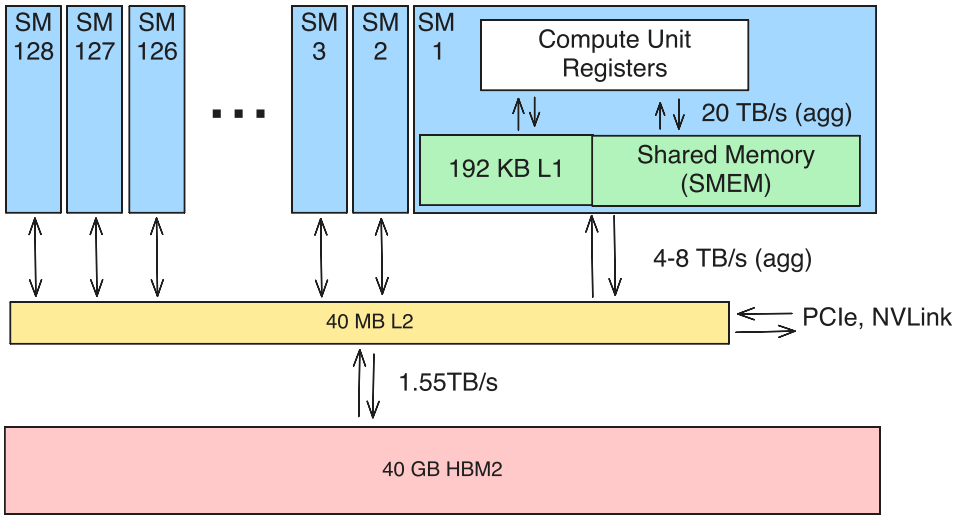


Figure 21: A100 memory hierarchy: registers, shared memory/L1, L2 cache, and HBM global memory.

Where CPUs use deep cache hierarchies to hide latency, GPUs prioritise bandwidth and expose a shallower hierarchy:

- **Registers:** single-cycle access, private per thread, scarcest on-chip resource (register count per kernel limits occupancy).
- **Shared memory:** on-chip, visible to all threads in a block; primary mechanism for explicit data reuse and inter-thread communication. ∼20 cycle latency.
- **L2 cache:** shared across all SMs, transparently caches global memory traffic (40 MB on A100). ∼200 cycle latency.
- **HBM (global memory):** bulk storage. A100-SXM4-40GB provides 40 GB at ∼1555 GB/s peak bandwidth. ∼500 cycle latency.

10.4.3 Memory Coalescing

When a warp executes a load/store, the hardware combines 32 individual thread addresses into as few 128-byte cache line transactions as possible. If consecutive threads access consecutive addresses (`base + i`), accesses are *coalesced*. Strided or scattered patterns require multiple transactions, wasting bandwidth.

Listing 69: Coalesced vs non-coalesced access

```
// Coalesced: consecutive threads read adjacent elements
float val = matrix[row * N + threadIdx.x];

// Non-coalesced: stride-N access
float val = matrix[threadIdx.x * N + col];
```

10.4.4 Shared Memory Bank Conflicts

Shared memory is divided into 32 banks (4 bytes wide, round-robin). When multiple warp threads access different words in the same bank, accesses serialise. Common fix: pad the leading dimension by one so stride is coprime to 32.

Listing 70: Bank conflict and the padding fix

```
__shared__ float tile[32][32];  // 32-way bank conflict on column access
__shared__ float tile[32][33];  // stride=33, conflicts eliminated
```

10.5 CUDA Programming Model

CUDA organises parallel execution into three levels: *grids*, *thread blocks*, and *threads*. A kernel launch creates one grid partitioned into blocks. Each block runs on one SM with access to shared memory and synchronisation barriers. Threads in different blocks cannot synchronise during kernel execution.

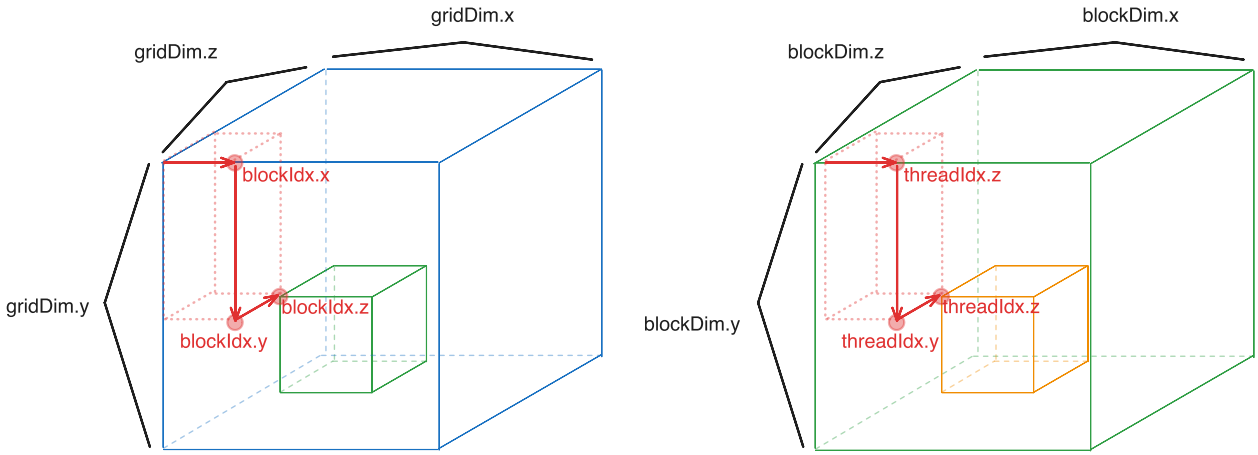


Figure 22: CUDA execution hierarchy: grids, thread blocks, and threads with built-in addressing variables.

10.5.1 Kernel Syntax and Launch Configuration

A kernel is declared with `__global__` and launched with triple-chevron syntax:

Listing 71: Vector addition kernel

```
__global__ void vecAdd(const float *a, const float *b, float *c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}

int threadsPerBlock = 256;
int blocksPerGrid = (n + threadsPerBlock - 1) / threadsPerBlock;
vecAdd<<<blocksPerGrid, threadsPerBlock>>>(d_a, d_b, d_c, n);
```

10.5.2 2D Grid Addressing

Listing 72: Matrix transpose with 2D grid

```
__global__ void transpose(const float *in, float *out, int M, int N) {
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    if (row < M && col < N)
        out[col * M + row] = in[row * N + col];
}

dim3 block(16, 16);
dim3 grid((N + 15) / 16, (M + 15) / 16);
transpose<<<grid, block>>>(d_in, d_out, M, N);
```

10.5.3 Block Size Selection

Upper limit: 1024 threads per block. Practical considerations:

- **Warp granularity:** block size should be a multiple of 32 (warp size) to avoid partially filled warps.
- **Occupancy:** each SM has finite registers, shared memory, and max 2048 resident threads. High per-thread register usage reduces resident warps.
- **Shared memory per block:** large allocations limit the number of co-resident blocks.

Common defaults: 128 or 256 threads. Performance-critical kernels tune empirically or use `__launch_bounds__`.

10.5.4 Shared Memory and Synchronisation

Shared memory is on-chip, programmer-managed, visible to all threads in a block. Two allocation modes:

Listing 73: Static vs dynamic shared memory

```
__shared__ float tile[TILE_SIZE][TILE_SIZE];         // static

extern __shared__ float smem[];                       // dynamic
kernel<<<grid, block, sharedBytes>>>(args);
```

10.5.5 Tiled Matrix Multiplication

The canonical shared memory pattern. Loads input tiles cooperatively, computes partial dot products, advances to next tile. Each loaded element is reused by all threads in its row/column, raising arithmetic intensity by the tile size factor.

Listing 74: Tiled matrix multiplication

```
#define TILE 16
__global__ void matmul(const float *A, const float *B, float *C,
                       int M, int N, int K) {
    __shared__ float As[TILE][TILE], Bs[TILE][TILE];
    int row = blockIdx.y * TILE + threadIdx.y;
    int col = blockIdx.x * TILE + threadIdx.x;
    float sum = 0.0f;

    for (int t = 0; t < (K + TILE - 1) / TILE; t++) {
        int aCol = t * TILE + threadIdx.x;
        int bRow = t * TILE + threadIdx.y;
        As[threadIdx.y][threadIdx.x] = (row < M && aCol < K)
                                       ? A[row * K + aCol] : 0.0f;
        Bs[threadIdx.y][threadIdx.x] = (bRow < K && col < N)
                                       ? B[bRow * N + col] : 0.0f;
        __syncthreads();
        for (int k = 0; k < TILE; k++)
            sum += As[threadIdx.y][k] * Bs[k][threadIdx.x];
        __syncthreads();
    }
    if (row < M && col < N) C[row * N + col] = sum;
}
```

For tile size T: global loads per output element drop from $2K$ to $2K/T$, and arithmetic intensity increases by factor T.

10.6 CPU–GPU Interaction

10.6.1 Kernel Launch Pipeline

A kernel launch goes through: (1) argument marshalling, (2) command enqueueing into the stream (host returns immediately), (3) driver serialisation to the GPU command buffer, (4) block distribution by the GigaThread engine to SMs with sufficient resources, (5) warp execution. Steps 1–4 cost ∼5–10 μs per launch on A100. For small kernels, launch overhead can dominate; the standard mitigation is *kernel fusion*.

10.6.2 Asynchronous Execution

Listing 75: Async pattern with streams and events

```
cudaStream_t s1, s2;
cudaEvent_t done1;
cudaStreamCreate(&s1); cudaStreamCreate(&s2);
cudaEventCreate(&done1);

cudaMemcpyAsync(d_in, h_in, bytes, cudaMemcpyHostToDevice, s1);
kernel1<<<grid1, block1, 0, s1>>>(d_in, d_tmp);
cudaEventRecord(done1, s1);

foo();  // host CPU work overlaps queued device work

cudaStreamWaitEvent(s2, done1, 0);
```

```
kernel2<<<grid2, block2, 0, s2>>>(d_tmp, d_out);
cudaMemcpyAsync(h_out, d_out, bytes, cudaMemcpyDeviceToHost, s2);
cudaStreamSynchronize(s2);
```

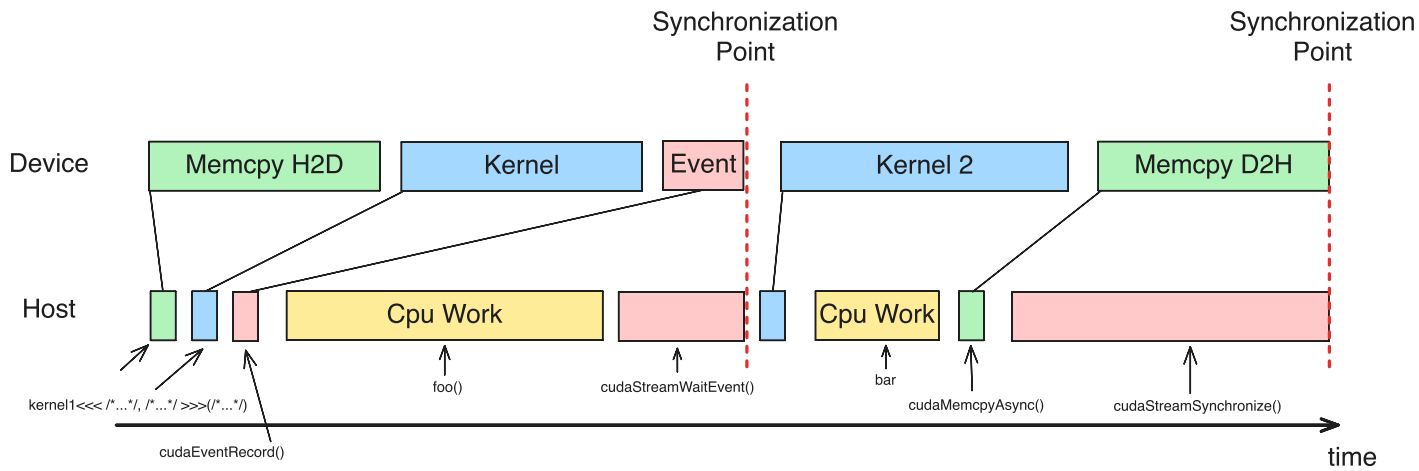


Figure 23: Asynchronous host–device timeline: host work overlaps queued device work until explicit synchronisation.

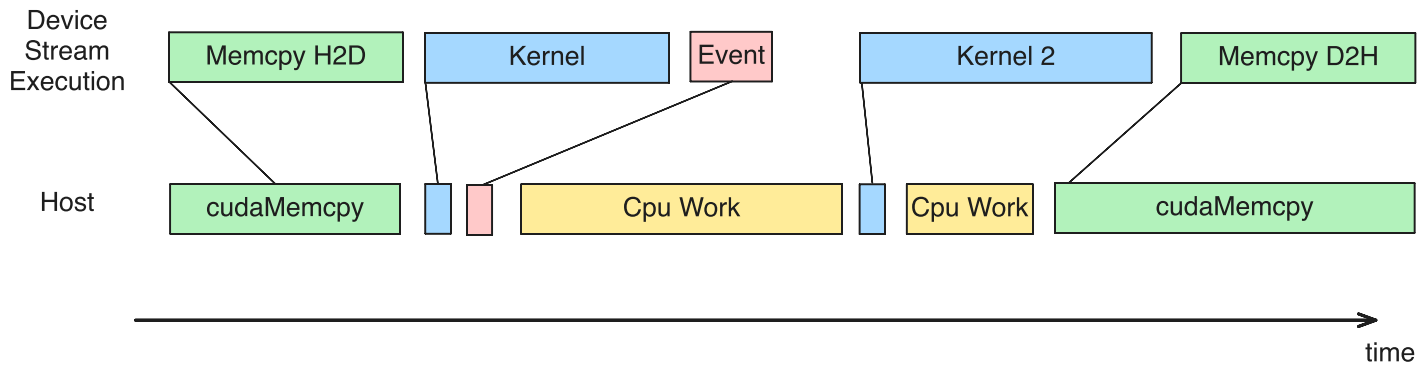


Figure 24: Synchronous timeline using blocking `cudaMemcpy`: host stalled during transfers.

For true async host-device copies, host buffers must be page-locked (pinned) via `cudaMallocHost`.

10.7 Benchmarking and Measurement

10.7.1 Sampling Benchmarks

10.7.2 Cache and Memory Profiling (perf, Valgrind)

10.7.3 Nsight Compute (GPU Profiling)

NVIDIA Nsight Compute collects hardware performance counters per kernel. Key metrics: achieved occupancy, memory throughput per hierarchy level, compute throughput, warp stall reasons, shared memory bank conflicts, and L2 hit rate.

11 Concurrency

11.1 C++ Memory Model

11.1.1 Atomics

There is only one atomic data type which is required to be lock free, namely `std::atomic_flag`. Pretty much every core C++ data type has an atomic version, for example

`std::atomic<bool>`, `std::atomic<int>`, `std::atomic<float>`, `std::atomic<double>`

It is important to check if those have hardware support (i.e. work lock free), this can be done by checking the compile time constant

`std::atomic<T>::is_always_lock_free`

11.1.2 Memory Ordering (`std::memory_order`)

The C++ memory model defines how memory accesses (reads and writes) from multiple threads can be observed relative to each other. By default, the compiler and CPU are free to reorder memory operations for performance; `std::memory_order` gives the programmer control over these guarantees.

- `memory_order_relaxed`: No ordering guarantees beyond atomicity. The operation is atomic but can be freely reordered with respect to other reads and writes. Useful for counters or flags where only the value itself matters, not the ordering relative to other data.
- `memory_order_acquire`: A *load* (read) with acquire semantics. Guarantees that no memory reads or writes *after* this load can be reordered *before* it. Typically used when reading a flag or lock to ensure that all data written by the releasing thread is visible.
- `memory_order_release`: A *store* (write) with release semantics. Guarantees that no memory reads or writes *before* this store can be reordered *after* it. Typically used when setting a flag or releasing a lock so that all prior writes are visible to the acquiring thread.
- `memory_order_acq_rel`: Combines acquire and release: a read-modify-write operation that is both an acquire (for the read part) and a release (for the write part).
- `memory_order_seq_cst`: Sequentially consistent ordering (the default). All threads observe all `seq_cst` operations in the same total order. Strongest guarantee, easiest to reason about, but potentially most expensive.

The fundamental pattern is the **acquire–release pair**: a store with `memory_order_release` in one thread *synchronises with* a load of the same atomic variable with `memory_order_acquire` in another thread. All writes that happened before the release are guaranteed to be visible after the acquire.

11.2 Thread Utilities (`std::this_thread`)

`std::this_thread::yield()` hints to the OS scheduler that the current thread is willing to give up its remaining time slice, allowing other threads to run. It does *not* guarantee a context switch; the scheduler may immediately reschedule the same thread. Useful in spinlocks to avoid burning CPU while waiting.

`std::this_thread::sleep_for(duration)` suspends the current thread for *at least* the specified duration (`std::chrono` based). Unlike `yield`, this is a hard pause; the thread will not be scheduled until the duration has elapsed (plus scheduling jitter).

Listing 76: std::this_thread utilities

```
std::this_thread::yield();                                  // hint: give up time slice
std::this_thread::sleep_for(std::chrono::milliseconds(10)); // sleep >= 10ms
```

11.3 Mutexes and Locks

11.3.1 Mutex Types

`std::mutex` (C++11) is the basic exclusive-ownership mutex. `std::timed_mutex` adds `try_lock_for`/`try_lock_until`. `std::recursive_mutex` allows the same thread to lock multiple times (must unlock the same number of times). `std::shared_mu` (C++17) supports shared (read) and exclusive (write) ownership for reader-writer patterns.

11.3.2 Lock Types

- `std::lock_guard<Mutex>` (C++11): simplest RAII lock. Locks on construction, unlocks on destruction. No manual lock/unlock after creation. Superseded by `scoped_lock` in C++17.
- `std::unique_lock<Mutex>` (C++11): more flexible. Supports deferred locking (`std::defer_lock`), timed locking, manual `lock()`/`unlock()`, and is *required* by `std::condition_variable::wait()`.
- `std::scoped_lock<Mutexes...>` (C++17): preferred default. Can lock *multiple* mutexes simultaneously with deadlock avoidance (uses `std::lock` internally). Non-movable, non-copyable.
- `std::shared_lock<SharedMutex>` (C++17): acquires shared (read) ownership. Multiple threads can hold a shared lock concurrently; exclusive locks block until all shared locks are released.

Listing 77: Lock types comparison

```
std::mutex m;

{ std::lock_guard<std::mutex> lg{m}; }       // C++11, single mutex
{ std::scoped_lock sl{m}; }                  // C++17, preferred replacement

std::mutex m1, m2;
{ std::scoped_lock sl{m1, m2}; }             // C++17, multiple mutexes

// C++11 equivalent for multiple mutexes:
std::lock(m1, m2);                            // lock both, deadlock-free
std::lock_guard lg1{m1, std::adopt_lock};
std::lock_guard lg2{m2, std::adopt_lock};

// unique_lock with deferred locking (required for condition variables)
std::unique_lock ulk{m, std::defer_lock};
ulk.lock();    // lock manually
ulk.unlock();  // unlock manually before destruction
```

11.3.3 Condition Variables

`std::condition_variable` allows threads to block until notified. Always use with a predicate to guard against *spurious wakeups* (the OS can wake a waiting thread without a notification). Requires `std::unique_lock` (not `lock_guard` or `scoped_lock`) because `wait()` must internally unlock the mutex while sleeping and re-lock before returning.

Listing 78: Condition variable pattern

```
std::mutex m;
std::condition_variable cv;
bool ready = false;

// Consumer
{
    std::unique_lock ulk{m};
    cv.wait(ulk, [&]{ return ready; }); // unlocks m, sleeps, re-locks on wake
    // ready == true here, m is locked
}

// Producer
{
    std::scoped_lock sl{m};
    ready = true;
}
cv.notify_one(); // or cv.notify_all()
```

11.4 Multiple Threads

11.5 Multiple Processes

11.5.1 OpenMP

11.6 Concurrent Data Structures

11.6.1 Mutex (Spinlock)

The only tricky part of implementing a basic mutex (spinlock) is keeping the acquire–release pair in mind regarding memory ordering; the actual logic is minimal. This tests the developer's understanding of the memory model (and that `.test()` is preferable to `.test_and_set()` for spinlocks, which on `atomic_flag` is only possible since C++20).

The skeleton is straightforward: `flag_` is an `std::atomic_flag`, the only atomic type guaranteed to be lock-free. Brace initialisation (value-initialised to clear) is valid since C++20. `lock()` spins until it acquires the flag, `unlock()` clears it. Unlocking an already-free mutex mirrors STL behaviour (no-op / no UB). Copy and move are deleted because a mutex represents a unique synchronisation point; copying or moving it would be meaningless. The destructor is explicitly defaulted since we declare the other special members.

Listing 79: Mutex skeleton

```
class Mutex
{
public:
    ~Mutex() = default;

    Mutex(const Mutex&) = delete;
    Mutex& operator=(const Mutex&) = delete;
    Mutex(Mutex&&) = delete;
    Mutex& operator=(Mutex&&) = delete;

    void lock();
    void unlock();
private:
    std::atomic_flag flag_{};
};
```

`lock()`: `test_and_set` is a read-modify-write operation (relatively expensive) that atomically sets the flag and returns the old value. We use `memory_order_acquire` so that, once we succeed, all writes made by the previous holder before their release-store are visible to us. If the flag was already set (lock held by someone else), we enter the spin body: rather than immediately retrying the expensive `test_and_set`, we probe with `test()` which is a read-only operation (cheaper, no cache-line bouncing). `memory_order_relaxed` suffices here because we are not yet acquiring; we are just waiting for the value to change. If the flag is still set we yield the time slice to avoid burning CPU.

Listing 80: Mutex::lock

```
void lock()
{
    while(flag_.test_and_set(std::memory_order_acquire)) {
        if(flag_.test(std::memory_order_relaxed)) {
            std::this_thread::yield();
        }
    }
}
```

`unlock()`: `clear` is a store that resets the flag. We use `memory_order_release` so that all writes we made while holding the lock are visible to the next thread that does an acquire-load of the same flag.

Listing 81: Mutex::unlock

```
void unlock()
{
    flag_.clear(std::memory_order_release);
}
```

11.6.2 SPSC (Single Producer Single Consumer)

A `DataWrapper<T>` pairs each work item with a sentinel flag signalling the last chunk. The `SPSC` class is templated on the data type and a callback invoked per item. It is non-copyable and non-movable since it owns a running thread. The destructor sets `exit_on_empty_` as a fallback (in case the sentinel was never pushed) and joins the consumer thread (`std::jthread` does not require a `joinable()` check).

Listing 82: SPSC skeleton

```
template <typename T>
struct DataWrapper
{
    T data{};
    bool is_last_chunk{false};
};

template <typename T, typename Callback>
class SPSC
{
public:
    SPSC(Callback callback);
    ~SPSC();

    SPSC(const SPSC&) = delete;
    SPSC& operator=(const SPSC&) = delete;
    SPSC(SPSC&&) = delete;
    SPSC& operator=(SPSC&&) = delete;

    void PushWork(const DataWrapper<T>& wrapper);
private:
    Callback callback_;
    std::queue<T> data_{};
    std::mutex mutex_data_{};
    std::jthread consumer_thread_;
    std::atomic<bool> exit_on_empty_{false};

    void Consume();
};
```

The constructor stores the callback (moved in) and immediately launches the consumer thread. The destructor sets the exit flag and joins.

Listing 83: SPSC constructor and destructor

```
SPSC(Callback callback)
    : callback_(std::move(callback))
    , consumer_thread_([this]{ Consume(); })
{}

~SPSC()
{
    exit_on_empty_ = true;
    consumer_thread_.join();
}
```

`PushWork`: Locks the queue, pushes the data, and if this is the last chunk, sets the exit flag so the consumer knows to drain and stop.

Listing 84: SPSC::PushWork

```
void PushWork(const DataWrapper<T>& wrapper)
{
    std::scoped_lock sl{mutex_data_};
    if(wrapper.is_last_chunk) {
        exit_on_empty_ = true;
    }
    data_.push(wrapper.data);
```

```
}
```

Consume: The consumer loop locks the queue, and if there is work it moves the front element out and pops. The lock is released *before* invoking the callback so the producer is not blocked while the (potentially slow) callback runs. If the queue is empty and the exit flag is set, we break. Otherwise we yield to avoid busy-spinning on an empty queue.

Listing 85: SPSC::Consume

```
void Consume()
{
    while(true) {
        std::unique_lock ulk{mutex_data_};
        if(!data_.empty()) {
            auto item = std::move(data_.front());
            data_.pop();
            ulk.unlock();
            callback_(item);
        } else if(exit_on_empty_) {
            break;
        } else {
            ulk.unlock();
            std::this_thread::yield();
        }
    }
}
```

11.6.3 Thread Pool

A thread pool pre-spawns a fixed number of worker threads that pull tasks from a shared queue. `WorkT` is a type alias for `std::function<void()>`. The pool is non-copyable (owning threads); move is also deleted for simplicity. The default constructor uses `std::thread::hardware_concurrency()` as the thread count.

Listing 86: ThreadPool skeleton

```
using WorkT = std::function<void()>;

class ThreadPool
{
public:
    ThreadPool();
    ThreadPool(std::size_t count);
    ~ThreadPool();

    ThreadPool(const ThreadPool&) = delete;
    ThreadPool& operator=(const ThreadPool&) = delete;
    ThreadPool(ThreadPool&&) = delete;
    ThreadPool& operator=(ThreadPool&&) = delete;

    size_t ThreadCount() const;

    template <typename Work>
    void submit(Work work);
private:
    std::atomic<bool> stop_{false};
    std::mutex mutex_;
    std::vector<std::jthread> threads_;
    std::queue<WorkT> work_queue_;

    auto Work() -> void;
};
```

The default constructor delegates to the parameterised one. Each thread runs the `Work()` loop. The destructor sets the stop flag and joins all threads; `std::jthread` does not require a `joinable()` check.

Listing 87: ThreadPool constructors and destructor

```
ThreadPool() : ThreadPool(std::thread::hardware_concurrency()) {}

ThreadPool(std::size_t count)
{
    threads_.reserve(count);
    for (std::size_t i{0zu}; i < count; ++i) {
        threads_.emplace_back([this] { Work(); });
    }
}

~ThreadPool()
{
    stop_ = true;
    for (auto& thread : threads_) {
        thread.join();
    }
}
```

`submit`: Locks the queue and pushes a type-erased `std::function<void()>` wrapping the caller's work. Templated so any callable can be submitted.

Listing 88: ThreadPool::submit

```
template <typename Work>
void submit(Work work)
{
    std::scoped_lock sl{mutex_};
    work_queue_.push([work] { work(); });
}
```

`Work`: Each worker thread runs this loop. It locks the queue, and if there is a task it moves it out and pops. The lock is *not* held during execution (the `scoped_lock` scope ends before the call). Exceptions from tasks are swallowed so one bad task does not kill a worker thread. If no task was available we yield to avoid busy-spinning.

Listing 89: ThreadPool::Work

```
auto Work() -> void
{
    std::optional<WorkT> func{};
    while (!stop_) {
        if (!func) {
            std::scoped_lock sl{mutex_};
            if (!work_queue_.empty()) {
                func = std::move(work_queue_.front());
                work_queue_.pop();
            }
        }
        if (func) {
            try {
                (*func)();
            } catch (...) {}
            func = std::nullopt;
        } else {
            std::this_thread::yield();
        }
    }
}
```

11.6.4 SPMC (Single Producer Multiple Consumer)

11.6.5 MPSC (Multiple Producer Single Consumer)

11.6.6 MPMC (Multiple Producer Multiple Consumer)

12 Operating Systems

12.1 Process, Thread

12.2 Scheduling

12.3 CPU Virtualisation

12.4 Memory Virtualisation

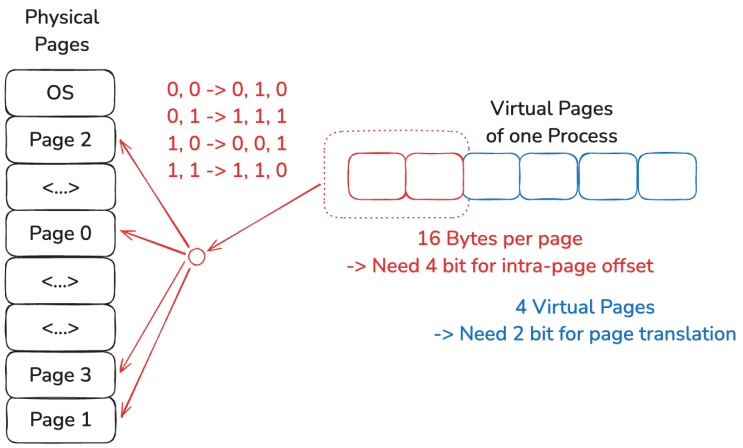


Figure 25: Virtual address translation: virtual page number + offset mapped to a physical frame number + offset

Paging as opposed to segmentation slices up the available virtual memory into fixed-size pieces.

13 Networking

13.1 OSI Model

13.2 UDP

13.3 TCP/IP

14 Trivia

14.1 Error #323 on GCC

There used to be a semi-famous issue where some compilers (notably GCC on x86) evaluated floating-point expressions using 80-bit x87 registers, which could lead to results that were "too correct" and therefore differ from other platforms and compilers (see Reference 15).

15 References

- Egor Suvorov.
  Using Floating-point in C++: What Works, What Breaks, and Why.
  CppCon 2025. `https://www.youtube.com/watch?v=m83TjrB6wYw`
- Klaus Iglberger, *C++ Software Design.* O'Reilly Media, 2022.
- Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces.* Arpaci-Dusseau Books. `https://pages.cs.wisc.edu/~remzi/OSTEP/`
- Fedor Pikus, *The Art of Writing Efficient Programs.* Apress, 2021.
- Anthony Williams, *C++ Concurrency in Action* (2nd ed.), Manning Publications, 2019.
- Glenford J. Myers, *The Art of Software Testing.* Wiley, 1979.
- Gabriel Staples, *C++ Mutexes and Locks.*
  `https://gabrielstaples.com/cpp-mutexes-and-locks/`